

Sistema de reconeixement automàtic de matrícules (ALPR)

Visió per Computador — Projecte final QP 2026

Marc Rams Estrada Aleix Vallhonrat Aragó

Juny de 2026

Índex

1	Introducció	3
1.1	Context i motivació	3
1.2	Objectiu	3
1.3	Restriccions i abast	4
1.4	Estructura de la memòria	4
2	Marc teòric	5
2.1	Preprocessament d'imatge	5
2.2	Detecció de vores: l'operador de Sobel	5
2.3	Morfologia matemàtica	6
2.4	Binarització	6
2.5	Descriptors de característiques: HOG	6
2.6	Classificadors entrenats des de zero	7
2.7	Mètriques d'avaluació	7
3	Disseny del sistema	9
3.1	Visió general del <i>pipeline</i>	9
3.2	Arquitectura modular i contractes entre fases	9
3.3	Decisions d'arquitectura clau	10
4	Implementació per mòduls	12
4.1	Mòdul 1 — Detector de matrícules	12
4.1.1	<i>Pipeline</i> de detecció	12
4.1.2	Filosofia de màxim recall	14
4.1.3	Alternatives descartades	14
4.2	Mòdul 2 — Alineació i segmentació de caràcters	14
4.2.1	Reptes dels retalls d'entrada	14
4.2.2	Correcció d'inclinació (deskew)	15
4.2.3	Segmentació per contorns	15
4.2.4	Criteri de rebuig de regions	16
4.2.5	Normalització de sortida a 28×28	16
4.3	Mòdul 3 — OCR (CNN entrenada des de zero)	17
4.3.1	Estratègia: fonts sintètiques i <i>domain gap</i>	17

4.3.2	Generació de dades	17
4.3.3	Arquitectura de la CNN	18
4.3.4	Entrenament	19
4.4	Mòdul 3b — Reader (muntatge final)	19
5	Experiments i resultats	20
5.1	Conjunt de dades i metodologia	20
5.2	Resultats del detector	20
5.2.1	Sensibilitat als llinars de forma	21
5.3	Resultats del segmentador i atribució d'errors	21
5.4	Resultats de l'OCR	22
5.5	Anàlisi del <i>domain gap</i>	23
5.6	Resultats end-to-end	24
5.7	Comparativa de tècniques	25
5.7.1	Detector: morfològic vs. cantonades vs. contorns	25
5.7.2	Segmentació: contorns vs. projecció vertical	26
5.7.3	Format de sortida 28×28 : vj vs. regla-or	26
5.7.4	Ablacions internes: NORM_H , skew i confiança	27
6	Discussió	29
6.1	Interpretació dels resultats per fase	29
6.2	Limitacions conegudes	30
6.3	Treball futur	30
7	Conclusions	31
	Referències	32
A	Enumeració de funcions	34
A.1	Funcions implementades per l'equip	34
A.2	Programari de tercers utilitzat	35
B	Codi font complet	38
B.1	Configuració global	38
B.2	Utilitats comunes	40
B.3	Fase 1 — Detector	44
B.4	Fase 2 — Segmentador	49
B.5	Fase 3 — OCR: generació de dades	64
B.6	Fase 3 — OCR: model, entrenament i avaluació	79
B.7	Fase 3b — Reader	97
B.8	Orquestrador end-to-end	98

Capítol 1

Introducció

1.1 Context i motivació

El reconeixement automàtic de matrícules (*Automatic License Plate Recognition*, ALPR) és una de les aplicacions clàssiques de la visió per ordinador, present en sistemes de control d'accessos, peatges, gestió d'aparcaments i vigilància del trànsit. El problema combina, en un sol *pipeline* de processament, diverses subtasques fonamentals de la disciplina: la localització d'un objecte dins d'una escena, la segmentació de les seves parts i el reconeixement de patrons.

Aquest projecte aborda la construcció d'un sistema ALPR complet a partir de fotografies de vehicles, amb una restricció metodològica central: el sistema s'ha de construir amb tècniques de visió clàssica i amb classificadors entrenats *des de zero* per l'equip, sense recórrer a cap model de detecció o reconeixement preentrenat. Aquesta restricció, lluny de ser un inconvenient, obliga a comprendre i justificar cada etapa del processament, que és precisament l'objectiu pedagògic del projecte.

1.2 Objectiu

L'objectiu és dissenyar, implementar i avaluar un sistema capaç de llegir el text d'una matrícula a partir de la imatge d'un vehicle. El sistema rep una fotografia i ha de retornar la cadena de caràcters de la matrícula, gestionant les dificultats pròpies d'imatges reals: variacions d'il·luminació, inclinacions lleugeres, diferents escales i la presència de regions que poden confondre's amb una matrícula.

Com a avanç dels resultats (capítol 5): el sistema final localitza el 98.1 % de les matrícules del conjunt d'avaluació, reconeix els caràcters ben segmentats amb una precisió del 89.63 %, i llegeix de manera completament correcta el 31.82 % de les matrícules, amb les pèrdues atribuïdes per fase: la segmentació n'és la primera font (rebuigs i recomptes de caràcters erronis), seguida

del salt del domini sintètic d'entrenament al real de l'OCR (*domain gap*).

1.3 Restriccions i abast

El projecte es desenvolupa sota les restriccions següents:

- **Sense models preentrenats.** La detecció i la segmentació es resolen amb processament d'imatge clàssic. El mòdul de reconeixement de caràcters pot ser una xarxa neuronal, però *entrenada per l'equip*, no un model descarregat ja entrenat.
- **Implementació pròpia justificada.** Cada tècnica triada s'ha de comparar amb les alternatives raonables i justificar amb evidència.
- **Reproductibilitat.** El sistema s'ha de poder executar sobre imatges noves, no incloses en el conjunt de desenvolupament.

L'abast cobreix tot el *pipeline* de lectura, des de la imatge d'entrada fins a la cadena de text de sortida, i la seva avaluació quantitativa fase a fase.

1.4 Estructura de la memòria

El Capítol 2 presenta el marc teòric de les tècniques emprades. El Capítol 3 descriu el disseny global del sistema i les decisions d'arquitectura. El Capítol 4 detalla la implementació de cada mòdul. El Capítol 5 recull els experiments i resultats. El Capítol 6 interpreta els resultats i les limitacions, i el Capítol 7 en sintetitza les conclusions. Els annexos contenen l'enumeració de funcions i el codi font complet.

Capítol 2

Marc teòric

Aquest capítol resumeix els fonaments de les tècniques emprades al llarg del sistema, organitzats segons l'ordre en què apareixen al *pipeline* de processament.

2.1 Preprocessament d'imatge

Abans de qualsevol anàlisi, la imatge es converteix a escala de grisos i es sotmet a operacions de millora. El **suavitzat gaussià** redueix el soroll convolucionant la imatge amb un nucli gaussià:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}, \quad (2.1)$$

on σ controla el grau de suavitzat. Un suavitzat previ és imprescindible abans de calcular gradients, perquè la derivació amplifica el soroll d'alta freqüència.

El **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*) [1] equalitza l'histograma de manera local, per regions, limitant el guany de contrast per evitar amplificar el soroll. És especialment útil amb il·luminació no uniforme, situació habitual en fotografies de vehicles (reflexos, ombres del para-xocs).

2.2 Detecció de vores: l'operador de Sobel

L'operador de Sobel [2] estima el gradient d'intensitat de la imatge mitjançant dos nuclis de convolució separables que aproximen les derivades parcials en x i en y :

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}. \quad (2.2)$$

La component S_x respon a les **vores verticals**. Aquesta propietat és clau per a la detecció de matrícules: el text d'una placa genera una densitat anormalment alta de vores verticals concentrades en una regió rectangular, un senyal que el fons, el para-xocs o els llums no produeixen amb la mateixa intensitat.

2.3 Morfologia matemàtica

Les operacions morfològiques transformen una imatge binària segons un element estructurant. Les dues operacions bàsiques són l'**erosió** ($\ominus$) i la **dilatació** ($\oplus$). A partir d'elles es defineixen:

$$\text{Obertura: } A \circ B = (A \ominus B) \oplus B, \quad (2.3)$$

$$\text{Tancament: } A \bullet B = (A \oplus B) \ominus B. \quad (2.4)$$

El **tancament** (*closing*) omple forats i connecta components propers; aplicat amb un element estructurant *horitzontal*, fusiona les vores verticals properes dels caràcters en un únic blob rectangular, que és el que permet localitzar la matrícula com una taca compacta. L'**obertura** (*opening*) elimina petites protuberàncies i soroll. L'etiquetatge de **components connexos** identifica després cada blob com una regió candidata. La Gràfic 4.1 (capítol 4) il·lustra aquest efecte sobre una imatge real del conjunt.

2.4 Binarització

La binarització separa primer pla i fons assignant a cada píxel un valor 0 o 1 segons un llindar. El **mètode d'Otsu** [3] tria un únic llindar global maximitzant la variància entre classes de l'histograma; funciona bé amb il·luminació uniforme, però falla quan una part de la imatge està en ombra i una altra il·luminada, perquè cap llindar global no serveix per a totes dues zones.

La **binarització adaptativa** resol aquest problema comparant cada píxel amb una estadística (per exemple, la mitjana) calculada en un veïnat local de mida `blockSize`, corregida per una constant C . Així s'adapta als gradients d'il·luminació de manera independent a cada regió, a costa de més sensibilitat al soroll. La tria entre Otsu i adaptativa és una de les decisions de disseny del segmentador (Capítol 3).

2.5 Descriptors de característiques: HOG

L'**histograma de gradients orientats** (*Histogram of Oriented Gradients*, HOG) [4] descriu una regió dividint-la en cel·les, calculant a cada cel·la un histograma d'orientacions del gradient i normalitzant per blocs per guanyar

robustesa davant la il·luminació. El vector resultant alimenta típicament un classificador SVM lineal sobre una finestra lliscant.

HOG és un detector vàlid per a objectes amb una forma característica, però presenta dos inconvenients per a la localització de matrícules: requereix *entrenament supervisat* amb molts exemples positius i negatius, i exigeix recórrer la imatge amb una finestra lliscant a múltiples escales, cosa computacionalment costosa. A més, és sensible a la rotació. Aquests motius el fan menys adequat que l'enfocament morfològic com a detector principal, tot i que es considera com a alternativa de comparació (Capítol 3).

2.6 Classificadors entrenats des de zero

Per al reconeixement de caràcters es consideren tres famílies de classificadors compatibles amb la restricció de "entrenats per l'equip": els **SVM** (separadors de marge màxim, sovint sobre descriptors HOG), els **k-NN** (classificació pel veïnatge més proper) i les **xarxes neuronals convolucionals** (CNN) [5]. Les CNN aprenen automàticament jerarquies de característiques visuals mitjançant capes de convolució, normalització i submostreig, i són especialment adequades per a la classificació d'imatges petites com els caràcters segmentats.

L'entrenament d'una CNN per a classificació multiclasse minimitza la pèrdua d'entropia creuada:

$$\mathcal{L} = - \sum_{c=1}^C y_c \log(\hat{y}_c), \quad (2.5)$$

on C és el nombre de classes, y_c l'etiqueta real (codificació one-hot) i $\hat{y}_c$ la probabilitat predita per la xarxa (sortida del *softmax*).

2.7 Mètriques d'avaluació

L'avaluació del sistema combina diverses mètriques segons la fase:

- **Recall** de detecció: proporció de matrícules reals localitzades per algun candidat. Es mesura de dues maneres: si el centre de la regió de referència (*ground truth*, GT) cau dins d'algun candidat, i amb el criteri més estricte de solapament **IoU**.
- **IoU** (*Intersection over Union*): mesura el solapament entre la caixa predita i la de referència, $\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$.
- **Accuracy per caràcter**: proporció de caràcters correctament reconeguts, mesurada de manera posicional (caràcters alineats) o mitjançant la *distància d'edició* (que penalitza insercions i supressions).

- **Accuracy per matrícula:** proporció de matrícules llegides completament correctes, la mètrica més exigent i la més representativa de l'utilitat real del sistema.
- **Matriu de confusió:** identifica quins parells de caràcters es confonen sistemàticament (per exemple $0 \leftrightarrow O$, $1 \leftrightarrow 7$), informació clau per diagnosticar el *domain gap* de l'OCR i decidir on actuar (augmentation de la Fase 3).

Capítol 3

Disseny del sistema

3.1 Visió general del *pipeline*

El sistema s'organitza en un *pipeline de tres fases en cadena*, amb una etapa final de lectura:

Foto $\rightarrow$ 1. Detector $\rightarrow$ 2. Segmentador $\rightarrow$ 3. OCR (CNN) $\rightarrow$ 3b. Reader $\rightarrow$ "M5X5X"

El **detector** (Secció 4.1) localitza regions candidates a contenir una matrícula dins de la imatge completa, amb una filosofia de màxim recall. El **segmentador** (Secció 4.2) parteix cada regió candidata en caràcters individuals i, alhora, filtra les regions que no són matrícules. L'**OCR** (Secció 4.3) classifica cada caràcter amb una CNN entrenada des de zero. Finalment, el **reader** (Secció 4.4) munta la cadena final netejant i concatenant la predicció de l'OCR, sense aplicar cap correcció contextual (el conjunt de dades no té format fix; vegeu Secció 4.4).

La Gràfic 3.1 il·lustra el *pipeline* complet sobre una imatge real del conjunt: el detector proposa els candidats, el segmentador normalitza i parteix el de la matrícula en caràcters 28×28 , i la CNN els classifica per compondre la lectura final. Noteu que el guió de la placa es rebutja correctament al filtre geomètric.

3.2 Arquitectura modular i contractes entre fases

Cada fase és un mòdul independent amb un contracte d'entrada/sortida net, cosa que permet substituir-la o avaluar-la per separat. El flux de dades i els tipus entre fases són:

- **Entrada del detector:** imatge en color (BGR, uint8).
- **Sortida del detector $\rightarrow$ entrada del segmentador:** una o més regions candidates (retalls de la imatge).

Pipeline complet sobre eu1: GT = M5XSX → lectura = M5XSX

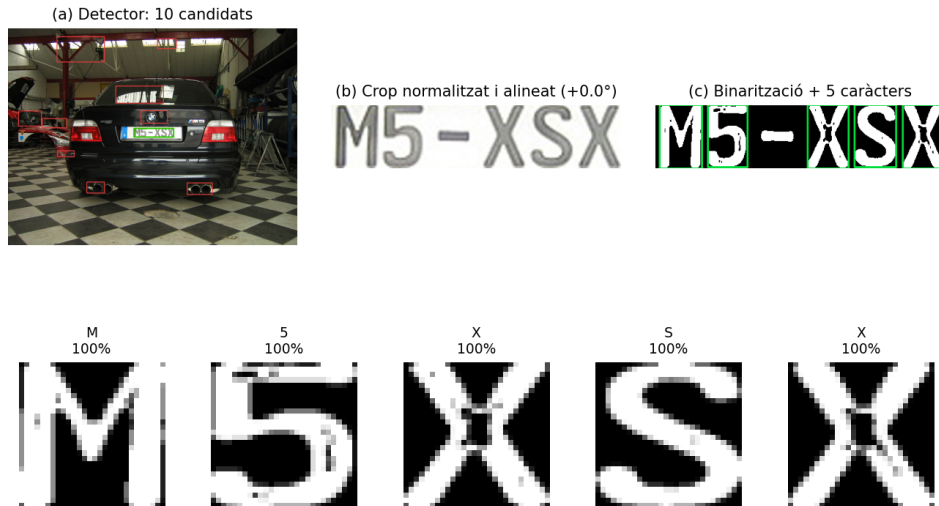


Figura 3.1: Exemple real del *pipeline* complet (imatge eu1): (a) candidats del detector (verd: el de la matrícula); (b) crop normalitzat i alineat; (c) binarització adaptativa amb els caràcters acceptats; a sota, els 28×28 amb la predicció i confiança de la CNN. Lectura final: M5XSX (exacta).

- **Sortida del segmentador → entrada de l'OCR:** una llista de caràcters normalitzats a 28×28 , binaris, caràcter blanc sobre fons negre; o bé una llista buida si la regió es rebutja com a no-matrícula.
- **Sortida de l'OCR → entrada del reader:** per a cada caràcter, la parella (classe predita, confiança).
- **Sortida del reader:** la cadena de text de la matrícula.

Aquesta separació en contractes és el que fa possible la comparació empírica sistemàtica de variants (per exemple, dues tècniques de segmentació o dos formats de sortida) sense tocar la resta del sistema.

3.3 Decisions d'arquitectura clau

Les decisions de disseny més rellevants es van prendre comparant alternatives i validant-les amb mesures sobre el conjunt de dades del projecte. Se'n resumeixen les principals; el detall de cadascuna és al capítol d'implementació.

- **Detector morfològic** (Sobel vertical + tancament horitzontal) enfront de Harris, HOG+SVM i Harris+DensityMap, per la seva simplicitat, robustesa a la perspectiva moderada i absència d'entrenament.
- **Segmentació per contorns** enfront de la projecció vertical, perquè no requereix conèixer el nombre de caràcters a priori i va validar més regions sobre el mateix conjunt de retalls.
- **Correcció d'inclinació per regressió de centroides** enfront de `minAreaRect` global, que tendia a estimar un angle nul.
- **OCR amb CNN entrenada amb fonts sintètiques** enfront d'un dataset de cal·ligrafia, per minimitzar el *domain gap* amb les fonts tipogràfiques reals de les matrícules.
- **Format de sortida 28×28 per redimensionat directe** enfront de la reconstrucció en un lleng amplí, decisió presa per mesura empírica directa amb la CNN.

Capítol 4

Implementació per mòduls

Tots els hiperparàmetres del sistema viuen en un únic fitxer de configuració (`alpr/config.py`); cap mòdul conté constants màgiques pròpies. Aquesta secció descriu cada fase amb els *valors exactes* que en regeixen el comportament. Les imatges es manipulen sempre en format BGR `uint8` (convenció d'OpenCV).

4.1 Mòdul 1 — Detector de matrícules

La detecció es fonamenta en una observació estructural: els caràcters d'una matrícula generen una **densitat anormalment alta de vores verticals** en una regió molt acotada. El detector explota aquesta propietat amb morfologia matemàtica, sense cap model entrenat. S'implementa com una cadena de quatre passos (`alpr/detector/`).

4.1.1 *Pipeline* de detecció

Pas 1 — Preprocessament (`preprocess.py`). La imatge es converteix a escala de grisos (`cv2.cvtColor`), se suavitza amb un filtre Gaussià de nucli 5×5 i $\sigma = 1,0$ (`cv2.GaussianBlur`) per atenuar el soroll, i se n'equalitza el contrast local amb CLAHE [1] (`cv2.createCLAHE`) amb `clipLimit = 2.0` i `tileGridSize = 8 \times 8`. El CLAHE local realça les vores dels caràcters fins i tot amb il·luminació desigual.

Pas 2 — Gradient vertical i binarització (`edges.py`). S'aplica l'operador de Sobel [2] *només en la direcció horitzontal* ($dx = 1$, $dy = 0$, `ksize = 3`), que respon als canvis d'intensitat al llarg de l'eix x , és a dir, a les vores *verticals*. La magnitud es porta a `uint8` (`cv2.convertScaleAbs`) i es binaritza amb el mètode d'Otsu [3] (`cv2.threshold` amb `THRESH_OTSU`), que tria el llindar global òptim automàticament.

Pas 3 — Morfologia i etiquetatge (`morphology.py`). Un *closing* amb element estructurant rectangular **horitzontal** (15×3) fusiona les vores verticals disperses en un blob compacte que cobreix la zona de text. Un *opening* posterior (5×5) elimina el soroll residual prim. Finalment, `cv2.connectedComponentsWithStats` (connectivitat 8) etiqueta cada blob i en calcula les estadístiques. La Gràfic 4.1 mostra l'efecte sobre una imatge real: les vores verticals disperses de la zona de text es fusionen en un únic blob compacte.

Pas 4 — Filtre de forma (`shape_filter.py`). Cada component connex es valida amb quatre criteris geomètrics (Taula 4.1); els que els superen es retornen com a bounding boxes (x, y, w, h). Els llindars són *deliberadament laxos* (filosofia de màxim recall, vegeu Secció 4.1.2).

Taula 4.1: Paràmetres del detector (`alpr/config.py`).

Etapa	Paràmetre	Valor
Suavitzat	nucli Gaussià / σ	5×5 / 1.0
CLAHE	<code>clipLimit</code> / <code>tileGridSize</code>	2.0 / 8×8
Sobel	<code>dx, dy</code> / <code>ksize</code>	1, 0 / 3
Binarització	mètode	Otsu global
Closing	element estructurant	15×3 (horitzontal)
Opening	element estructurant	5×5
Filtre de forma	amplada / alçada mínimes	40 px / 10 px
	àrea relativa a la imatge	[0,001, 0,10]
	<i>aspect ratio</i> w/h	[1,5, 9,0]
	<i>extent</i> àrea/($w \cdot h$)	$\geq 0,25$

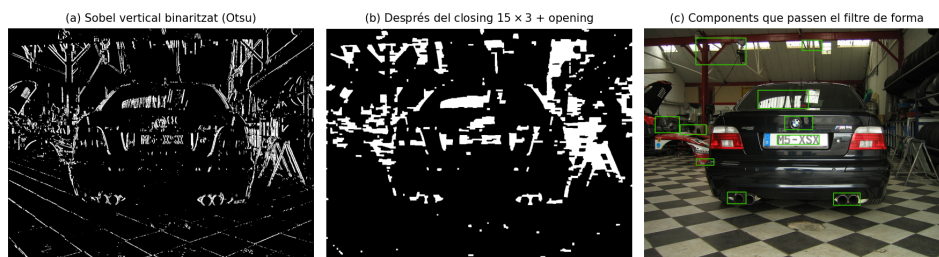


Figura 4.1: Efecte de la morfologia sobre una imatge real: (a) el Sobel vertical binaritzat deixa vores disperses; (b) el *closing* horitzontal 15×3 fusiona les de la zona de text en un blob compacte; (c) components connexos que superen el filtre de forma.

4.1.2 Filosofia de màxim recall

El detector prioritza **no perdre cap matrícula** per damunt de la precisió: és preferible retornar diversos candidats (alguns falsos positius) i garantir que la matrícula real sempre hi sigui, deixant que el segmentador (Secció 4.2) filtri la resta. Aquesta tria es valida als resultats: el detector captura el 98.1 % de les matrícules amb només 3.5 candidats per imatge de mitjana (Secció 5.2).

4.1.3 Alternatives descartades

Abans d'adoptar el detector morfològic es van implementar i comparar altres tècniques clàssiques de localització, totes avaluades amb el mateix criteri de recall sobre les 108 imatges anotades (Taula 5.6):

- **Detecció de cantonades de Harris** [6] amb mapa de densitat: agrupa zones amb alta densitat de cantonades. Recall inferior i molt més soroll (mediana de 6 candidats per imatge).
- **Canny multiescala** [7]: detecció de contorns rectangulars a diferents escales. El menys robust dels tres en aquest conjunt.
- **HOG + SVM** [4]: descriptor de gradients orientats amb un classificador lineal. Es va descartar perquè exigeix entrenament supervisat (contra l'esperit del projecte) i un mostreig negatiu acurat.

El detector morfològic va resultar alhora el *més simple* (sense entrenament) i el *de més recall*, per la qual cosa es va adoptar com a solució oficial.

4.2 Mòdul 2 — Alineació i segmentació de caràcters

El segmentador rep un retall candidat i en parteix el text en caràcters individuals 28×28 ; si el retall no és plausiblement una matrícula, retorna una llista buida. El *pipeline* complet per retall és:

normalitza $\rightarrow$ deskew $\rightarrow$ binaritzat $\rightarrow$ contorns $\rightarrow$ filtre
 $\rightarrow$ fila dominant $\rightarrow$ dedup. $\rightarrow$ rebuig $\rightarrow$ exporta 28×28 .

4.2.1 Reptes dels retalls d'entrada

Els retalls del detector presenten tres dificultats: (i) il·luminació no uniforme (ombres, reflexos), que descarta un llindar global i obliga a binarització adaptativa; (ii) inclinació per la perspectiva de la foto, que cal corregir abans de partir; i (iii) falsos positius del detector (banda blava de la UE, text del

concessionari), que cal rebutjar. A més, les matrícules d'aquest conjunt són molt petites (alçada de 18 px a 46 px al *ground truth*), cosa que motiva la normalització d'escala prèvia.

Normalització d'escala (`normalize.py`). Abans de detectar res, el retall es reescala a una alçada de treball comuna `NORM_H = 96` px preservant l'*aspect ratio* (`cv2.resize`, `INTER_CUBIC` en ampliar). El motiu és que la binarització adaptativa usa un `blockSize` fix de 31 px: sense normalitzar, en una placa de 18 px el bloc és més gran que el caràcter sencer i la binarització falla i sobre-segmenta. La tria de 96 px es justifica empíricament a Taula 5.9.

4.2.2 Correcció d'inclinació (`deskew`)

La inclinació es corregeix per **regressió lineal sobre els centroides** dels caràcters candidats (`deskew.py`). Com que els caràcters d'una matrícula estan alineats horitzontalment per disseny, el pendent m de la recta $c_y = m c_x + b$ ajustada als seus centroides (amb `numpy.polyfit`) revela directament la inclinació de la placa. L'angle és:

$$\alpha = \arctan(m) \cdot \frac{180}{\pi}. \quad (4.1)$$

El crop es rota al voltant del seu centre (`cv2.getRotationMatrix2D` + `cv2.warpAffine`, `INTER_CUBIC`, `BORDER_REPLICATE` per evitar franges negres als cantons). Es descarta la rotació si hi ha menys de 3 centroides (`DESKEW_MIN_CANDIDATS`, senyal insuficient) o si $|\alpha| > 15^\circ$ (`DESKEW_ANGLE_MAX`, detecció errònia).

Esquema de dues passades. La detecció de candidats per estimar l'angle (passada 1) usa un filtre geomètric *tolerant*; la segmentació final sobre el crop ja alineat (passada 2) usa el filtre *estricte*. Això evita que un filtre massa rígid en un crop encara inclinat deixi sense prou centroides per estimar bé l'angle.

4.2.3 Segmentació per contorns

Binarització adaptativa (`binarize.py`). Sobre el crop alineat i en grisos s'aplica `cv2.adaptiveThreshold` amb mètode `ADAPTIVE_THRESH_MEAN_C`, polaritat `THRESH_BINARY_INV` (caràcters blancs sobre fons negre), `blockSize = 31` i $C = 15$. Un `MORPH_CLOSE` amb nucli **vertical** 1×3 consolida els traços fins (com la 'I') sense fusionar caràcters veïns. *Aquests paràmetres són idèntics als de l'augmentació del dataset sintètic* (Secció 4.3.2), de manera que la textura de vores que veu la CNN en inferència coincideix amb la de l'entrenament.

Extracció i filtratge (contours.py). Es busquen els contorns externs amb `cv2.findContours` en mode `RETR_EXTERNAL` [8] i aproximació `CHAIN_APPROX_SIMPLE`. `RETR_EXTERNAL` ignora els forats interiors (de la ‘O’, ‘0’, ‘D’, ‘B’...), evitant comptar-los com a caràcters separats. Els bounding boxes resultants passen per tres etapes (Taula 4.2):

1. **Filtre geomètric groller** relatiu a l'alçada de la placa `h_placa` (mai en píxels absoluts): alçada dins $[0,30, 1,05] \cdot h_{\text{placa}}$, *aspect ratio* dins $[0,05, 1,10]$ (el límit inferior 0.05 preserva la ‘I’/‘1’), i àrea $\geq 0,015 \cdot h_{\text{placa}}^2$ (elimina *speckle*).
2. **Selecció de la fila dominant:** es conserven els blobs amb alçada dins $[1 \pm 0,30]$ de la mediana, robust a soroll d’altres files.
3. **Deduplicació per IoU** ($\geq 0,50$): davant de solapaments, es conserva el bbox de més àrea (el contorn exterior).

4.2.4 Criteri de rebuig de regions

La funció `is_plausible_plate` (`validate.py`) decideix si el conjunt de caràcters detectats és una matrícula real; si no, es rebutja tot el retall. Aplica quatre comprovacions, totes relatives (mai en píxels absoluts):

1. **Recompte** dins $[5, 9]$ caràcters (matrícules EU del conjunt).
2. **Uniformitat d'alçades:** $\sigma_h/h_{\text{med}} \leq 0,20$.
3. **Alineació de la baseline:** $\sigma_{c_y}/h_{\text{med}} \leq 0,25$.
4. **Cobertura horitzontal:** $(x_{\text{max}} - x_{\text{min}})/W \geq 0,30$.

Aquest criteri és el mecanisme que **filtra els falsos positius del detector**: una banda blava de la UE o un text de concessionari no superen aquestes restriccions de coherència de fila.

4.2.5 Normalització de sortida a 28×28

Cada caràcter acceptat s’exporta a 28×28 en el format `vj` (`char_export.py`): retall *tight* de la imatge binària de detecció seguit d’un redimensionat directe a 28×28 amb `INTER_AREA`, que estira l’*aspect ratio* i omple el caràcter de vora a vora. La sortida són caràcters blancs sobre fons negre, la mateixa convenció que el dataset d’entrenament. L’alternativa avaluada (format `regla-or`, que reconstrueix el glif en un llenç 64×64) es compara empíricament a Secció 5.7: la mesura, contraintuïtiva, va donar clarament guanyador el format simple `vj`.

Taula 4.2: Paràmetres del segmentador (`alpr/config.py`). Tots els llindars geomètrics són relatius a l'alçada de la placa o a la mediana.

Etapa	Paràmetre	Valor
Normalització	<code>NORM_H</code>	96 px
Deskew	mín. centroides / $ \alpha _{\max}$	3 / 15°
Binarització	<code>blockSize</code> / C	31 / 15
	<code>MORPH_CLOSE</code>	1 × 3 (vertical)
Filtre geomètric	alçada (rel. h_{placa})	[0,30, 1,05]
	<i>aspect ratio</i> w/h	[0,05, 1,10]
	àrea mínima (rel. h_{placa}^2)	0,015
Fila dominant	banda (rel. mediana)	[0,70, 1,30]
Deduplicació	llindar IoU	0,50
Criteri de rebuig	nombre de caràcters	[5, 9]
	σ_h/h_{med}	$\leq 0,20$
	$\sigma_{c_y}/h_{\text{med}}$	$\leq 0,25$
	cobertura horitzontal	$\geq 0,30$

4.3 Mòdul 3 — OCR (CNN entrenada des de zero)

4.3.1 Estratègia: fonts sintètiques i *domain gap*

No es disposa d'un conjunt prou gran de caràcters reals etiquetats per entrenar un classificador des de zero. La solució és generar un **dataset 100 % sintètic** a partir de fonts tipogràfiques: les matrícules s'imprimeixen amb tipografies, no a mà, de manera que una font sintètica reproduïx molt millor el domini real que un dataset de cal·ligrafia. El repte que això introdueix és el ***domain gap*** (la diferència entre el domini d'entrenament i el d'inferència): la xarxa entrena amb glifs nets i ha d'inferir sobre fotografies degradades. Tota l'augmentation (Secció 4.3.2) està dissenyada precisament per estrènyer aquest salt.

4.3.2 Generació de dades

La generació és una seqüència de dos passos.

Render de fonts (`01_render_fonts.py`). Per a cadascuna de les 19 fonts `.ttf/.otf` de `resources/fonts/` i per a cada una de les **36 classes** (A–Z, 0–9) es renderitza el caràcter centrat en un llenç de 64×64 px amb marge de 6 px, en negre sobre blanc (Pillow: `ImageFont`, `ImageDraw`). La mida de font s'ajusta per encaixar el glif dins l'àrea útil. Són les imatges base

netes, sense degradar.

Augmentation (02_augment.py). Cada imatge base genera mostres degradades aplicant, *en ordre estricte*, quatre grups de transformacions (Taula 4.3). El darrer grup, la binarització, fa servir **exactament** els mateixos paràmetres que el segmentador real (`adaptiveThreshold`, `MEAN_C`, `BINARY_INV`, 31, 15); és el pas que garanteix que la textura de vores d’entrenament i d’inferència coincideixi. La reducció final a 28×28 (`INTER_NEAREST`) es deixa per al final per no escapar el glif. Es generen 500 mostres per classe, amb partició 80/20 entre entrenament i validació (`seed = 42`), és a dir $36 \times 500 = 18\,000$ imatges (400 d’entrenament i 100 de validació per classe).

Taula 4.3: Grups d’augmentation (02_augment.py). El glif és negre sobre blanc fins al grup D, que l’inverteix.

Grup	Transformació	Rang / probabilitat
A — geometria	rotació	$\pm 8^\circ$ ($p = 0,7$)
	shear	$\pm 0,1$ ($p = 0,4$)
	translació	$\pm 10\%$ ($p = 0,5$)
	escala	$[0,85, 1,15]$ ($p = 0,5$)
B — traç	traç prim / gruixut	nucli 2-3 ($p = 0,3$ c.u.)
	soroll sal i pebre	$\leq 3\%$ píxels ($p = 0,4$)
C — contorn	retall d’un costat	0-3 px ($p = 0,3$)
	descentrat	± 3 px ($p = 0,4$)
D — binarització	<code>adaptiveThreshold</code>	<code>MEAN_C/INV/31/15</code> (sempre)

4.3.3 Arquitectura de la CNN

El classificador és `CharCNN (model.py)`, una xarxa convolucional compacta de **831 332 paràmetres** que rep un tensor $1 \times 28 \times 28$ normalitzat a $[-1, 1]$ i produeix logits de 36 classes. L’arquitectura, inspirada en les xarxes convolucionals clàssiques [5], encadena dos blocs convolucionals i un classificador dens:

$$\underbrace{1 \times 28 \times 28}_{\text{entrada}} \rightarrow \text{Conv}(1 \rightarrow 32) \rightarrow \text{Conv}(32 \rightarrow 64) \rightarrow \underbrace{64 \times 7 \times 7}_{3136} \rightarrow \text{FC}(256) \rightarrow \text{FC}(36)$$

Cada bloc convolucional és $\text{Conv}(3 \times 3, \text{padding} = 1) \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}(2)$, amb normalització per lots (*batch normalization*, BN) [9], i redueix la resolució de 28 a 14 i de 14 a 7. Després del segon bloc hi ha un `Dropout(0,4)` [10]; el classificador és $\text{FC}(3136 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow$

Dropout(0,3) $\rightarrow$ FC(256 $\rightarrow$ 36). La capa de sortida dona logits crus: la funció de pèrdua d'entropia creuada (Equació (2.5)) ja incorpora la *softmax*.

4.3.4 Entrenament

L'entrenament (`train.py`) minimitza l'entropia creuada (Equació (2.5), amb $C = 36$ classes) amb l'optimitzador Adam [11] (*learning rate* 10^{-3}), *batch size* 64, fins a 30 *epochs* amb *early stopping* de paciència 5 sobre la precisió de validació (`seed=42` per a reproductibilitat). Es desa el millor checkpoint (`models/char_cnn_best.pth`). El millor model es va obtenir a l'*epoch* 20, amb una **precisió de validació del 98.11 %** (Secció 5.4). El checkpoint guarda també el mapa `class_to_idx`, de manera que la inferència (`infer.py`) reconstrueix exactament la mateixa correspondència classe $\leftrightarrow$ índex.

4.4 Mòdul 3b — Reader (muntatge final)

El reader (`reader.py`) munta la cadena final amb una única operació: concatena els caràcters predits, els passa a majúscules i, amb una expressió regular, n'elimina els '?' (baixa confiança) i qualsevol símbol que no sigui una lletra (A-Z) o un dígit (0-9).

Per què NO s'aplica correcció contextual. Les matrícules d'aquest conjunt **no segueixen cap patró**: poden ser de qualsevol país europeu (Sèrbia, Eslovènia, Turquia, Espanya...), amb lletres i dígitos en majúscules, de 5 a 9 caràcters i en *ordre totalment aleatori*. Cap posició té un tipus esperat (lletra o dígit), de manera que no es pot saber la direcció de cap confusió ('S' $\leftrightarrow$ '5', 'O' $\leftrightarrow$ '0'...): corregir a cegues només introduiria errors nous.

Capítol 5

Experiments i resultats

5.1 Conjunt de dades i metodologia

L'avaluació es fa sobre **108 imatges anotades** (`data/raw/`): cada imatge té un fitxer de *ground truth* amb la bounding box de la matrícula i la seva transcripció correcta en majúscules. El reconeixement s'entrena, en canvi, sobre el dataset **sintètic** de 18 000 imatges (Secció 4.3.2); cap caràcter real s'usa per entrenar, només per avaluar.

Cada fase es mesura amb la mètrica adequada al seu objectiu:

- **Detector:** *recall* de la matrícula, en dues variants — que el centre del GT caigui dins d'algun candidat, i que existeixi un candidat amb $\text{IoU} \geq 0,5$ amb el GT— més el nombre de candidats per imatge.
- **Segmentador / sistema:** atribució d'errors per fase (*funnel*) i recompte de matrícules llegides perfectament.
- **OCR:** *char accuracy* (per caràcter) i *plate accuracy* (matrícula completa correcta). La comparació amb el GT es fa per **distància d'edició** (Levenshtein [12]), robusta a desalineacions de longitud, i també **posicionalment** sobre les matrícules ben segmentades.

5.2 Resultats del detector

El detector morfològic captura **98.1 %** de les matrícules pel criteri de centre (106 de 108), amb només 3.5 candidats per imatge de mitjana (Taula 5.1). És exactament el comportament que busca la filosofia de màxim recall: gairebé cap matrícula es perd a l'origen, a canvi d'uns pocs falsos positius que el segmentador filtrarà. El *gap* fins al recall amb $\text{IoU} \geq 0,5$ (79.6 %) indica que la caixa no sempre és perfectament ajustada, cosa que la Fase 2 corregeix amb la seva pròpia normalització i deskew.

Taula 5.1: Resultats del detector sobre les 108 imatges amb *ground truth*.

Mètrica	Valor
Recall (centre del GT dins d'algun candidat)	98.1 % (106/108)
Recall (IoU $\geq 0,5$ amb el GT)	79.6 % (86/108)
Candidats per imatge (mediana / mitjana / màx)	3 / 3,5 / 11

5.2.1 Sensibilitat als lindars de forma

El compromís recall $\leftrightarrow$ candidats es controla amb els lindars del filtre de forma (Taula 5.2). Afluixar l'*extent* mínim o pujar l'*aspect ratio* màxim augmenta el recall però genera més candidats; estrènyer-los fa el contrari. Els valors adoptats (**EXTENT_MIN** = 0,25, **ASPECT_RATIO_MAX** = 9,0) maximitzen el recall mantenint els candidats acotats.

Taula 5.2: *Sweep* dels lindars de forma del detector (un knob cada cop).

Paràmetre	Valor	Recall (centre)	Recall (IoU)	Cand. mitjà
EXTENT_MIN	0,15	98.1 %	80.6 %	3,49
	0,25	98.1 %	79.6 %	3,45
	0,35	97.2 %	79.6 %	3,24
	0,45	94.4 %	79.6 %	2,65
ASPECT_RATIO_MAX	5,0	70.4 %	55.6 %	2,94
	7,0	97.2 %	79.6 %	3,38
	9,0	98.1 %	79.6 %	3,45
	12,0	98.1 %	79.6 %	3,51

5.3 Resultats del segmentador i atribució d'errors

La millor manera d'entendre el sistema és seguir cada matrícula pel pipeline i veure *on* es perd (Taula 5.3). De les 108 matrícules, el detector només en perd 2. El segmentador en rebutja 27 senceres (plaques reals descartades pel criteri de rebuig) i, de les 79 que accepta, 29 més es perden per **recompte de caràcters erroni** (sobre/infra-segmentació): per molt perfecte que fos l'OCR, aquestes lectures mai coincidirien amb el GT. Només 22 imatges fallen per **confusió pura de l'OCR** (longitud correcta, caràcter mal classificat).

La conclusió de l'atribució és clara: **el segmentador és la primera font de pèrdues del sistema** —56 de 108 imatges (51.9 %), entre rebutjos i recomptes erronis—, més del doble que la confusió pura de l'OCR (20.4 %). Una lectura ingènua del funnel que atribuís a l'OCR tot el que falla després de l'acceptació n'hauria sobreestimat la culpa (47.2 %): el desglossament per

longitud mostra que 29 de les 51 lectures errònies són, en realitat, errors de segmentació.

Taula 5.3: Atribució d’errors end-to-end sobre les 108 imatges, desglossada per la fase realment responsable (una lectura de longitud incorrecta és un error de *segmentació*, no de classificació).

Fase responsable	On es perd / resultat	Imatges	%
—	Lectura perfecta	28	25.9 %
Segmentador	Recompte de caràcters erroni	29	26.9 %
Segmentador	Placa real rebutjada	27	25.0 %
OCR	Confusió de caràcters (longitud bona)	22	20.4 %
Detector	No detectada	2	1.9 %
Funnel: 106 detectades → 79 segmentades → 50 recompte correcte → 28 perfectes			

Avaluat sobre *tots* els candidats del detector (no només la caixa GT), el segmentador valida **88 grups-matrícula** amb transcripció disponible, que són la base de les mètriques d’OCR de la secció següent. La sobre-segmentació és l’error de longitud dominant: de les 88, 50 tenen el nombre exacte de caràcters, mentre que 29 en generen de més (típicament soroll de la banda de la UE) i 9 de menys (Gràfic 5.1).

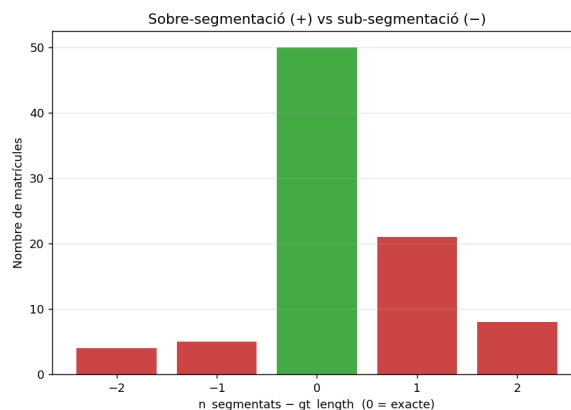


Figura 5.1: Diferència entre el nombre de caràcters segmentats i el del GT (0 = exacte). Domina la sobre-segmentació (+1, +2).

5.4 Resultats de l’OCR

La CNN aprèn les fonts sintètiques gairebé a la perfecció: la millor precisió de validació és del **98.11 %** a l’*epoch* 20 (Gràfic 5.2). Sobre caràcters *reals*, la precisió per caràcter de les matrícules ben segmentades és del **89.63 %**

(311/347 caràcters alineats); incloent-hi els errors de longitud via distància d'edició, el *char accuracy* baixa al 73.69 % i la *plate accuracy* (matrícula totalment correcta) és del 31.82 % (28/88) — Taula 5.4.

Taula 5.4: Resultats del reconeixement de caràcters (CharCNN, 36 classes).

Mètrica	Valor
Validació sintètica (<i>epoch</i> 20)	98.11 %
Char accuracy real (posicional, alineat)	89.63 % (311/347)
Char accuracy real (distància d'edició)	73.69 %
Plate accuracy (matrícules perfectes)	31.82 % (28/88)

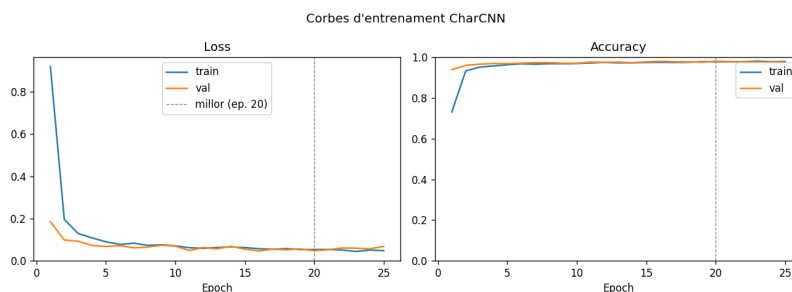
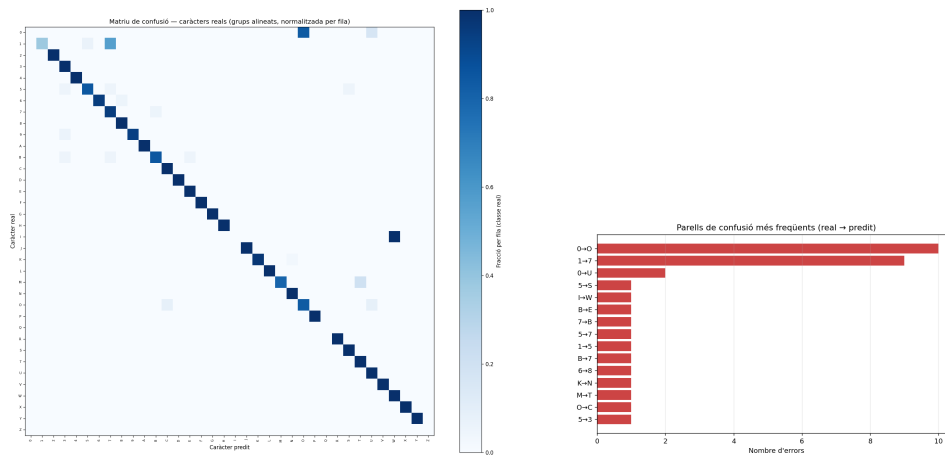


Figura 5.2: Corbes d'entrenament de la CharCNN (loss i accuracy, train vs. validació). El millor model és a l'*epoch* 20.

Errors freqüents. La matriu de confusió sobre caràcters reals (Gràfic 5.3a) concentra els errors en pocs parells visualment ambigus: els dos més habituals són $0 \rightarrow O$ (10 casos) i $1 \rightarrow 7$ (9 casos), seguits de lluny per $0 \rightarrow U$, $5 \rightarrow S$ i altres (Gràfic 5.3b). Són confusions de *forma*, pròpies de la *domain gap*, no errors de segmentació.

5.5 Anàlisi del *domain gap*

El *domain gap* es quantifica comparant la precisió per caràcter en el domini sintètic i en el real. Sobre *reconeixement pur* (caràcters ben segmentats), la caiguda és de la validació sintètica (98.11 %) a la real posicional (89.63 %), és a dir $\approx 8,5$ punts. La caiguda més gran fins al 73.69 % (distància d'edició) *no* és reconeixement: incorpora també els errors de longitud de la segmentació (sobre-segmentació de la Gràfic 5.1). És a dir, una part del que sovint s'atribueix sencer al «domain gap» és, de fet, soroll de segmentació — l'atribució de la Taula 5.3 ho confirma: 29 de les 51 lectures errònies són errors de recompte, no de classificació.



(a) Matriu de confusió (normalitzada per fila). (b) Parells de confusió més freqüents.

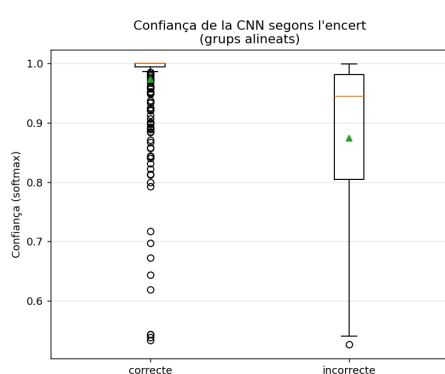
Figura 5.3: Errors de reconeixement sobre caràcters reals (grups alineats).

De fet, la frontera entre «error de segmentació» i «error d'OCR» és **difusa**, perquè la precisió de la CNN depèn fortament del crop que el segmentador li lliura. Les ablacions ho mostren directament: canviar *només* el format d'exportació del segmentador gairebé dobla el char accuracy (Taula 5.8), i variar només l'alçada de normalització el mou 16 punts (Taula 5.9) — amb la mateixa CNN i els mateixos pesos en tots els casos. La «precisió de l'OCR» és, doncs, una propietat conjunta del parell segmentador + CNN, i una part de les confusions residuals pot venir de la qualitat del caràcter segmentat (p. ex. l'estirament de l'*aspect ratio* del format vj pot acostar un '1' a un '7'). N'hi ha, però, d'intrínsecament ambigües: la confusió més freqüent, $0 \rightarrow O$, és irresoluble a nivell de glif (en moltes tipografies de matrícula són idèntics) i sense patró posicional no es pot desambiguar (Secció 4.4).

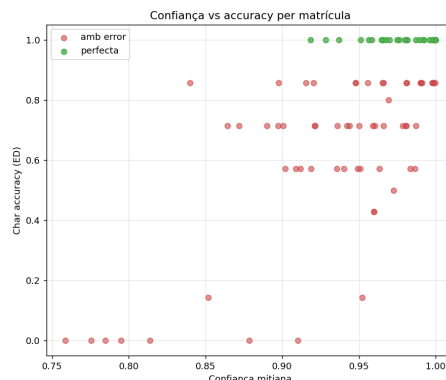
Un símptoma característic del domain gap és la **sobre-confiança**: la xarxa, entrenada amb glifs nets, assigna confiança alta fins i tot quan s'equivoca. La confiança mediana global és de 0.96, i les prediccions *incorrectes* mantenen una confiança elevada (Gràfic 5.4), de manera que un simple llindar de confiança no n'elimina la majoria (vegeu Secció 5.7.4).

5.6 Resultats end-to-end

La Taula 5.5 i la Gràfic 5.5 mostren exemples qualitatius del pipeline complet. El sistema llegeix correctament les matrícules netes i frontals; els errors residuals són les confusions del *domain gap* ja descrites ($O \leftrightarrow 0$, $S \leftrightarrow 5$) i caràcters extra provinents de la banda blava de la UE.



(a) Confiança segons l'encert.



(b) Confiança mitjana vs. precisió per matrícula.

Figura 5.4: Sobre-confiança de la CNN: els errors no tenen confiança especialment baixa.

Taula 5.5: Exemples qualitatius de lectura end-to-end (crops reals a la Gràfic 5.5).

Matrícula (GT)	Lectura	Comentari
M5XSX	M5XSX	perfecta (conf. 100 %); el guió es rebutja
WA56660	WAS6660	S↔5, O↔0
BS47040	XBS47040	placa correcta dins (O↔0) + banda UE

5.7 Comparativa de tècniques

Aquesta secció recull les decisions de disseny preses **per mesura empírica**, comparant cada alternativa amb la solució adoptada sobre el mateix conjunt. Són el nucli de la metodologia del projecte: cap decisió es justifica per intuïció, sinó per la seva prestació mesurada.

5.7.1 Detector: morfològic vs. cantonades vs. contorns

Taula 5.6: Tècniques de detecció (recall sobre les 108 imatges).

Tècnica	Recall (centre)	Recall (IoU)	Cand. med.
Morfològic (adoptat)	98.1 %	79.6 %	3
Harris + mapa de densitat	82.4 %	38.9 %	6
Canny multiescala	73.1 %	61.1 %	5



Figura 5.5: Crops reals (normalitzats i alineats) dels tres exemples de la Taula 5.5, amb el GT i la lectura del sistema.

5.7.2 Segmentació: contorns vs. projecció vertical

La segmentació per contorns és **netament superior** a la projecció vertical sobre aquest conjunt: no només valida moltes més regions, sinó que la lectura resultant és incomparablement millor (Taula 5.7). La projecció, sensible a caràcters tocant-se i a soroll de columna, gairebé no produeix lectures correctes.

Taula 5.7: Segmentació: contorns enfront de projecció vertical (mateix pipeline).

Tècnica	ROIs	Char (ED)	Plate	Char (pos.)
Contorns (adoptada)	88	73.69 %	31.82 %	89.63 %
Projecció vertical	33	3.51 %	0.0 %	0.0 %

5.7.3 Format de sortida 28×28 : vj vs. regla-or

És la decisió més **contraintuïtiva** del projecte. La intuïció teòrica deia que calia replicar el procés d'entrenament (reconstruir el glif en un llenç 64×64 abans de binaritzar: format **regla-or**). La mesura va demostrar el contrari: el redimensionat directe del retall binari (vj) gairebé **dobla** el *char accuracy* (Taula 5.8). Es va adoptar vj per evidència empírica directa.

Taula 5.8: Format del caràcter 28×28 : redimensionat directe (**vj**) enfront de reconstrucció en llenç (**regla-or**). Mateixes 88 ROIs i mateixa CNN.

Format	Char acc. (ED)	Plate acc.	Char acc. (pos.)
vj (resize directe)	73.69 %	31.82 %	89.63 %
regla-or (llenç 64×64)	38.56 %	10.23 %	59.75 %

5.7.4 Ablacions internes: **NORM_H**, skew i confiança

Normalització d'escala (Taula 5.9). L'alçada de treball és crítica: amb **NORM_H** = 32 la plate accuracy és del 10 %, i amb 96 (el valor adoptat) puja al 31.8 %. Per sobre de 96 les plaques grans es difuminen i torna a baixar. Confirma la decisió D9 de 96 px.

Taula 5.9: *Sweep* de **NORM_H** (alçada de normalització del retall).

NORM_H	ROIs	Char acc. (ED)	Plate acc.	Char acc. (pos.)
32	69	58.84 %	10.14 %	66.82 %
48	80	73.84 %	28.75 %	82.87 %
64	79	74.68 %	22.78 %	81.98 %
96	88	73.69 %	31.82 %	89.63 %
128	87	70.03 %	20.69 %	81.67 %
160	88	67.22 %	18.18 %	78.23 %

Deskew (Taula 5.10). En aquest conjunt, força frontal, el skew per centroides té un efecte modest sobre el *char accuracy*, però és **net-positiu**: accepta 4 ROIs més (88 vs. 84) i dona la millor *plate accuracy*. L'alternativa **minAreaRect**, no millora cap mètrica rellevant i accepta menys ROIs.

Taula 5.10: Variants de correcció d'inclinació.

Variant	ROIs	Char acc. (ED)	Plate acc.	Perfectes
Centroides (adoptat)	88	73.69 %	31.82 %	28
Sense skew	84	74.14 %	27.38 %	23
minAreaRect (rebutjat)	84	74.49 %	30.95 %	26

Llindar de confiança (Taula 5.11). El llindar adoptat (0,5) és l'òptim: filtra els '?' menys fiables sense descartar prediccions bones. Pujar-lo a 0,9 elimina massa caràcters correctes (efecte de la sobre-confiança, Secció 5.5) i enfonsa la plate accuracy.

Taula 5.11: *Sweep* del llindar de confiança de l'OCR.

Llindar	Char acc. (ED)	Plate acc.	Perfectes
0,0	71.57 %	27.27 %	24
0,3	71.73 %	27.27 %	24
0,5	73.69 %	31.82 %	28
0,7	72.88 %	27.27 %	24
0,9	69.44 %	15.91 %	14

Capítol 6

Discussió

6.1 Interpretació dels resultats per fase

El comportament del sistema s'entén millor analitzant cada fase pel seu objectiu propi. El detector compleix la seva funció — no perdre cap matrícula — amb un recall molt alt en termes de centre del GT, a costa de generar alguns candidats falsos que les fases posteriors han de filtrar. El segmentador actua simultàniament com a partididor de caràcters i com a filtre de falsos positius; aquesta segona funció, però, té un cost que l'atribució d'errors fa explícit: el criteri de rebuig també descarta una de cada quatre matrícules *reals*.

L'anàlisi de la Taula 5.3 mostra que el sistema té **dos colls d'ampolla acoblats**, no un de sol. El primer, i la font més gran de fallades observades, és el **segmentador**: entre plaques reals rebutjades i recomptes de caràcters erronis li són atribuïbles el 51.9 % de les imatges, més del doble que la confusió pura de l'OCR (20.4 %). El segon és el **reconeixement**: fins i tot amb el recompte correcte, la precisió per caràcter del 89.63 % es multiplica al llarg de la matrícula (per a 7 caràcters, $0,8963^7 \approx 0,46$), perquè una matrícula només és correcta si ho són *tots* els seus caràcters.

Cap de les dues fases, perfeccionada per separat, desbloqueja el sistema: amb un OCR perfecte i el segmentador actual, la *plate accuracy* màxima seria del ≈ 57 % (les 50 ROIs de 88 amb recompte exacte); amb una segmentació perfecta i l'OCR actual, del ≈ 47 %. A més, com s'ha vist a Secció 5.5, la frontera entre totes dues és difusa: la precisió de la CNN depèn del format i l'escala del crop que rep, de manera que part del que es comptabilitza com a error de reconeixement és, en realitat, qualitat del caràcter segmentat. El *domain gap* sintètic→real ($\approx 8,5$ punts posicionals) hi continua contribuint —amb confusions intrínsecament ambigües com $0 \leftrightarrow O$ que cap segmentació pot resoldre—, però no és, per si sol, el coll d'ampolla del sistema.

6.2 Limitacions conegudes

- **Falsos positius del detector.** Regions com la banda blava de la Unió Europea o el text del concessionari sota la placa generen caràcters addicionals que el sistema no sempre filtra. És sobretot una responsabilitat de la primera fase.
- ***Domain gap* de l'OCR.** La xarxa s'entrena amb fonts netes i interfereix sobre fotografies reals degradades. Els errors més freqüents són confusions entre caràcters visualment semblants.
- **Plaques molt petites.** Quan la matrícula original ocupa pocs píxels, la informació de partida és insuficient i la segmentació arrossega soroll tot i la normalització d'escala.
- **Absència de format fix.** Com que les matrícules del conjunt no segueixen cap patró (5–9 caràcters europeus en ordre aleatori), el reader no pot validar ni corregir la lectura per posició; es limita a muntar la predicció de l'OCR. Recuperar les confusions sistemàtiques és, per tant, feina de la Fase 3 (reduir el *domain gap*), no de la lectura.

6.3 Treball futur

Les línies de millora prioritàries, ordenades per impacte esperat segons l'atribució d'errors (Taula 5.3), són: (1) reduir les pèrdues del **segmentador**, la primera font de fallades: d'una banda el recompte erroni de caràcters (filtrar millor la banda blava de la UE i els blobs fusionats, que expliquen la sobre-segmentació de la Gràfic 5.1), i de l'altra el rebuig de plaques reals (revisar els llindars del criteri de rebuig, que avui descarta una de cada quatre); (2) reduir el *domain gap* de l'OCR amb *augmentation* més realista o amb un ajust fi amb caràcters reals etiquetats; i (3) afinar el detector per reduir els falsos positius en origen i ajustar les caixes (millor IoU), cosa que alleugeriria la feina del segmentador.

Capítol 7

Conclusions

S’ha dissenyat, implementat i avaluat un sistema complet de reconeixement automàtic de matrícules construït íntegrament amb tècniques pròpies, sense recórrer a cap model de detecció o reconeixement preentrenat. El sistema combina una detecció morfològica basada en gradient vertical, una segmentació de caràcters per contorns amb correcció d’inclinació prèvia, i un reconeixement amb una xarxa neuronal convolucional entrenada des de zero amb dades sintètiques generades per l’equip.

Cada decisió de disseny rellevant s’ha pres comparant alternatives i validant-les amb mesures sobre el conjunt de dades del projecte, i no a partir d’intuïcions a priori. Aquesta metodologia ha donat resultats en alguns casos contraintuïtius però empíricament superiors, com en la tria del format de sortida dels caràcters.

El detector compleix el seu objectiu gairebé del tot (captura pràcticament totes les matrícules), i la xarxa reconeix amb una precisió alta els caràcters ben segmentats. Els reptes que queden oberts són dos i estan acoblats: d’una banda, la **robustesa del segmentador**, que és la primera font de fallades del sistema (entre rebutjos de plaques reals i recomptes de caràcters erronis, li és atribuïble la meitat de les imatges perdudes); de l’altra, el salt del domini sintètic d’entrenament al domini real de les fotografies (*domain gap*), que limita el reconeixement fins i tot amb una segmentació de longitud correcta. L’anàlisi d’ablacions mostra, a més, que totes dues fases no són independents: la precisió de la CNN depèn fortament del format i l’escala del caràcter que el segmentador li lliura.

En xifres globals, el sistema captura el 98.1 % de les matrícules a la detecció, en reconeix els caràcters ben segmentats amb una precisió del 89.63 %, i llegeix de manera *completament correcta* 28 de les 88 matrícules avaluades (31.82 % de *plate accuracy*). El marge de millora queda repartit, de manera mesurada, entre dos fronts: la segmentació (rebuig i recompte, la primera font de pèrdues) i el *domain gap* de l’OCR.

Referències

- [1] K. Zuiderveld, “Contrast Limited Adaptive Histogram Equalization,” a *Graphics Gems IV*, Academic Press, 1994, pàg. 474 - 485.
- [2] I. Sobel i G. Feldman, “A 3x3 Isotropic Gradient Operator for Image Processing,” a *Stanford Artificial Intelligence Project (SAIL)*, 1968.
- [3] N. Otsu, “A Threshold Selection Method from Gray-Level Histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, núm. 1, pàg. 62 - 66, 1979.
- [4] N. Dalal i B. Triggs, “Histograms of Oriented Gradients for Human Detection,” a *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005.
- [5] Y. LeCun, L. Bottou, Y. Bengio i P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, vol. 86, núm. 11, pàg. 2278 - 2324, 1998.
- [6] C. Harris i M. Stephens, “A Combined Corner and Edge Detector,” a *Proceedings of the Alvey Vision Conference*, 1988, pàg. 147 - 151.
- [7] J. Canny, “A Computational Approach to Edge Detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, núm. 6, pàg. 679 - 698, 1986.
- [8] S. Suzuki i K. Abe, “Topological Structural Analysis of Digitized Binary Images by Border Following,” *Computer Vision, Graphics, and Image Processing*, vol. 30, núm. 1, pàg. 32 - 46, 1985.
- [9] S. Ioffe i C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” a *International Conference on Machine Learning (ICML)*, 2015.
- [10] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever i R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pàg. 1929 - 1958, 2014.
- [11] D. P. Kingma i J. Ba, “Adam: A Method for Stochastic Optimization,” a *International Conference on Learning Representations (ICLR)*, 2015.

- [12] V. I. Levenshtein, “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” *Soviet Physics Doklady*, vol. 10, núm. 8, pàg. 707-710, 1966.
- [13] OpenCV team, *OpenCV: Open Source Computer Vision Library*, <https://opencv.org>, 2024.
- [14] C. R. Harris, K. J. Millman, S. J. van der Walt et al., “Array programming with NumPy,” *Nature*, vol. 585, núm. 7825, pàg. 357-362, 2020.
- [15] A. Paszke, S. Gross, F. Massa et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” a *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [16] TorchVision maintainers and contributors, *TorchVision: PyTorch’s Computer Vision library*, <https://github.com/pytorch/vision>, 2024.
- [17] A. Clark i Pillow contributors, *Pillow: The friendly PIL fork*, <https://python-pillow.org>, 2024.
- [18] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, núm. 3, pàg. 90-95, 2007.
- [19] F. Pedregosa, G. Varoquaux, A. Gramfort et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pàg. 2825-2830, 2011.

Apèndix A

Enumeració de funcions

Aquest annex distingeix, com exigeix l'enunciat, les funcions **implementades per l'equip** de les **aportades per llibreries de tercers**. El criteri és estricte: una funció és «pròpia» si el seu cos l'ha escrit l'equip, encara que internament cridi primitives externes; les primitives externes (p. ex. `cv2.adaptiveThreshold`) es comptabilitzen com a programari de tercers, mai com a pròpies. El codi complet és a l'Capítol B.

A.1 Funcions implementades per l'equip

Taula A.1: Funcions pròpies — utilitats comunes (`alpr/common/`).

Funció	Fitxer	Descripció
<code>load_image / save_image</code>	<code>io.py</code>	càrrega/desat d'imatge BGR
<code>to_grayscale / iter_images</code>	<code>io.py</code>	conversió a grisos; iterador de directori
<code>iou</code>	<code>geometry.py</code>	<i>intersection over union</i> de dos bboxes
<code>nms</code>	<code>geometry.py</code>	supressió de no-màxims
<code>parse_annotation / load_annotations</code>	<code>annotations.py</code>	parseig del GT (bbox + matrícula)
<code>parse_crop_filename</code>	<code>annotations.py</code>	extreu (<code>stem</code> , <code>idx</code>) del nom de retall
<code>parse_char_filename</code>	<code>annotations.py</code>	extreu (<code>stem</code> , <code>box</code> , <code>char</code>) del nom

Taula A.2: Funcions pròpies — detector (`alpr/detector/`).

Funció	Fitxer	Descripció
<code>preprocess</code>	<code>preprocess.py</code>	grisos + Gaussià + CLAHE
<code>sobel_vertical_binary</code>	<code>edges.py</code>	Sobel vertical + binarització Otsu
<code>morphology_and_label</code>	<code>morphology.py</code>	closing/opening + components connexos
<code>filter_by_shape</code>	<code>shape_filter.py</code>	filtre geomètric de candidats
<code>detect_boxes / detect</code>	<code>detector.py</code>	API: bboxes / retalls de candidats
<code>detect_debug</code>	<code>detector.py</code>	API amb etapes intermèdies

Taula A.3: Funcions pròpies — segmentador (`alpr/segmenter/`).

Funció	Fitxer	Descripció
<code>normalitza_escala</code>	<code>normalize.py</code>	reescala el retall a <code>NORM_H</code>
<code>estimate_skew_angle</code>	<code>deskew.py</code>	angle per regressió de centroides
<code>deskew</code>	<code>deskew.py</code>	rotació (passada 1 tolerant)
<code>binarize_adaptive</code>	<code>binarize.py</code>	binarització adaptativa + close
<code>extract_contours</code>	<code>contours.py</code>	bboxes amb <code>RETR_EXTERNAL</code>
<code>filter_geometric</code>	<code>contours.py</code>	filtre relatiu a l'alçada de la placa
<code>select_dominant_row</code>	<code>contours.py</code>	selecció de la fila per mediana
<code>remove_overlapping</code>	<code>contours.py</code>	deduplicació per IoU
<code>detecta_projeccio</code>	<code>projection.py</code>	segmentació alternativa per projecció
<code>is_plausible_plate</code>	<code>validate.py</code>	criteri de rebuig de ROI
<code>preprocessa_caracter</code>	<code>char_export.py</code>	exporta un caràcter 28×28 (vj/regla-or)
<code>crops_and_resize</code>	<code>char_export.py</code>	exporta tots els caràcters
<code>segment / segmenta_caixa</code>	<code>segmenter.py</code>	API: caràcters / etapes completes
<code>save_chars</code>	<code>segmenter.py</code>	desa els caràcters a disc

A.2 Programari de tercers utilitzat

Les funcions de la Secció A.1 criden internament les primitives de les llibreries següents, que *no* són obra de l'equip i es referencien aquí (Taula A.6). Cap altre programari de tercers s'utilitza al pipeline; en particular, **no** s'usa `scikit-image`.

Taula A.4: Funcions pròpies — OCR i reader (`alpr/ocr/`, `alpr/reader/`, `notebooks/ocr/`).

Funció / classe	Fitxer	Descripció
<code>CharCNN</code>	<code>model.py</code>	arquitectura de la CNN (36 classes)
<code>build_data loaders</code>	<code>train.py</code>	càrrega de <code>train/val</code>
<code>train_one_epoch / evaluate</code>	<code>train.py</code>	bucles d'entrenament i validació
<code>train / plot_curves</code>	<code>train.py</code>	entrenament complet; corbes
<code>load_model</code>	<code>infer.py</code>	càrrega del checkpoint
<code>predict / predict_char</code>	<code>infer.py</code>	inferència (array / fitxer)
<code>filter_by_confidence</code>	<code>infer.py</code>	separació per llindar de confiança
<code>edit_distance</code>	<code>eval_real.py</code>	distància de Levenshtein (pròpia)
<code>group_files / evaluate_group</code>	<code>eval_real.py</code>	agrupació i avaluació per ROI
<code>compute_metrics / plot_confusion</code>	<code>eval_real.py</code>	mètriques i matriu de confusió
<code>find_fonts / fit_font / render_char</code>	<code>01_render_fonts.py</code>	render de glifs des de fonts
<code>apply_geometric / apply_stroke</code>	<code>02_augment.py</code>	augmentation (grups A, B)
<code>apply_contour / apply_binarize</code>	<code>02_augment.py</code>	augmentation (grups C, D)
<code>augment / build_dataset</code>	<code>02_augment.py</code>	orquestració i construcció del dataset
<code>read_plate</code>	<code>reader.py</code>	muntatge de la cadena final (sense correcció)

Taula A.5: Funcions pròpies — orquestrador (`main.py`).

Funció	Fitxer	Descripció
<code>process_image</code>	<code>main.py</code>	pipeline complet sobre una imatge
<code>draw_results</code>	<code>main.py</code>	anotació visual dels resultats
<code>run</code>	<code>main.py</code>	bucle sobre imatge o directori + CSV

Taula A.6: Programari de tercers i primitives concretes utilitzades.

Llibreria	Primitives utilitzades	Referència
OpenCV (<code>cv2</code>)	<code>cvtColor</code> , <code>GaussianBlur</code> , <code>createCLAHE</code> , <code>Sobel</code> , <code>threshold</code> (<code>Otsu</code>), <code>getStructuringElement</code> , <code>morphologyEx</code> , <code>connectedComponentsWithStats</code> , <code>adaptiveThreshold</code> , <code>findContours</code> , <code>boundingRect</code> , <code>getRotationMatrix2D</code> , <code>warpAffine</code> , <code>resize</code> , <code>imread/imwrite</code>	[13]
NumPy	operacions d'array, <code>polyfit</code> , <code>median</code> , <code>std</code> , <code>convolve</code> , <code>where</code>	[14]
PyTorch (<code>torch</code> , <code>torch.nn</code>)	<code>Conv2d</code> , <code>BatchNorm2d</code> , <code>ReLU</code> , <code>MaxPool2d</code> , <code>Dropout</code> , <code>Linear</code> , <code>Sequential</code> , <code>CrossEntropyLoss</code> , <code>Adam</code> , <code>softmax</code> , <code>save/load</code>	[15]
torchvision	<code>datasets.ImageFolder</code> , <code>transforms</code> (<code>Grayscale</code> , <code>Resize</code> , <code>ToTensor</code> , <code>Normalize</code>)	[16]
Pillow (PIL)	<code>Image</code> , <code>ImageDraw</code> , <code>ImageFont.truetype</code> , <code>getbbox</code> (render de fonts)	[17]
Matplotlib	<code>pyplot</code> (corbes d'entrenament, matriu de confusió)	[18]
scikit-learn	<code>metrics.confusion_matrix</code> (només a <code>eval_real.py</code>)	[19]

Apèndix B

Codi font complet

Aquest annex inclou el codi font complet del sistema. Els fitxers estan organitzats en subcarpetes segons les fases del pipeline, i es mostren en l'ordre d'execució. El codi està comentat i documentat per facilitar la comprensió de les decisions de disseny i implementació.

B.1 Configuració global

```
1 """
2 config.py
3 =====
4 Única font d'hiperparàmetres i rutes del projecte ALPR.
5 Cap mòdul ha de tenir constants màgiques pròpies: tot passa
6     per aquí.
7 """
8
9 from pathlib import Path
10
11 # -- Rutes -----
12 # Ancorades a l'arrel del projecte (alpr/config.py →
13 #     parents[1]), no al cwd, perquè
14 # el pipeline funcioni des de qualsevol directori d'execució.
15 ROOT_DIR = Path(__file__).resolve().parents[1]
16 DATA_RAW_DIR = ROOT_DIR / "data" / "raw"
17 DATA_PROCESSED_DIR = ROOT_DIR / "data" / "processed"
18 DATA_CHARS_DIR = ROOT_DIR / "data" / "chars"
19 DATA_SYNTHETIC_DIR = ROOT_DIR / "data" / "synthetic"
20 MODELS_DIR = ROOT_DIR / "models"
21 MODEL_CNN_PATH = MODELS_DIR / "char_cnn_best.pth"
22 OUTPUT_DIR = ROOT_DIR / "output"
23 FONTS_DIR = ROOT_DIR / "resources" / "fonts"
24
25 IMAGE_EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp", ".tif",
26     ".tiff"}
```

```

25 # -- Fase 1 - Detector -----
26 # Preprocessament
27 CLAHE_CLIP_LIMIT = 2.0
28 CLAHE_TILE_SIZE = (8, 8)
29 GAUSS_KSIZE = (5, 5)
30 GAUSS_SIGMA = 1.0
31
32 # Morfologia (mides pensades per a imatges ~400-600 px
    d'amplada;
33 # augmentar per a resolucions superiors)
34 CLOSE_KERNEL = (15, 3)
35 OPEN_KERNEL = (5, 5)
36
37 # Filtre de forma
38 AREA_RATIO_MIN = 0.001 # 0.1 % de la imatge
39 AREA_RATIO_MAX = 0.10 # 10 % de la imatge
40 ASPECT_RATIO_MIN = 1.5
41 ASPECT_RATIO_MAX = 9.0
42 EXTENT_MIN = 0.25
43 MIN_WIDTH = 40
44 MIN_HEIGHT = 10
45
46 # -- Fase 2 - Segmentador -----
47 NORM_H = 96 # alçada de normalització del
    crop abans de detectar
48 # (D9/ADR-002: les plaques del
    dataset són de 18-46 px i
49 # blockSize=31 falla sense
    escalar a una alçada de
    treball comuna)
50 ADAPT_BLOCK = 31 # blockSize per a
    adaptiveThreshold
51 ADAPT_C = 15 # constant C per a
    adaptiveThreshold
52 MORPH_CLOSE_KERNEL = (1, 3) # consolida traços fins (p.
    ex. 'I') sense fusionar veïns
53
54 DESKEW_MIN_CANDIDATS = 3 # mínim de centroides per
    estimar l'angle (ADR-003)
55 DESKEW_ANGLE_MAX = 15.0 # graus: si |angle| supera, no
    rota
56
57 H_CHAR_MIN_REL = 0.30 # h_min = H_CHAR_MIN_REL ·
    h_placa (filtre groller)
58 H_CHAR_MAX_REL = 1.05 # h_max = H_CHAR_MAX_REL ·
    h_placa
59 AR_MIN = 0.05 # aspect ratio mínim (cobreix
    la 'I'/'1', AR real 0.05-0.12)
60 AR_MAX = 1.10 # aspect ratio màxim (per
    sobre sol ser soroll o fusions)
61 AREA_MIN_REL = 0.015 # àrea mínima del blob /
    h_placa2 (elimina speckle)

```



```

62
63 H_MED_TOL      = 0.30          # banda de la fila dominant:
    [1±H_MED_TOL]·mediana d'alçades
64 ROW_STD_H_MAX  = 0.20          # std_h / h_med màxima
    (uniformitat)
65 ROW_STD_CY_MAX = 0.25          # std_cy / h_med màxima
    (alineació baseline)
66 WIDTH_OCC_MIN  = 0.30          # cobertura horitzontal mínima
    del crop
67
68 N_CHARS_MIN     = 5            # rang de caràcters vàlids per
    a una matrícula
69 N_CHARS_MAX     = 9            # (matrícules EU del dataset)
70 IOU_OVERLAP     = 0.50         # llindar de deduplicació IoU
71
72 CANVAS_SIZE     = 64           # mida del llenç per als
    renders de font
73 MARGIN          = 6            # marge interior al llenç
    (píxels)
74 OUTPUT_SIZE     = 28           # mida de sortida del caràcter
    per a la CNN
75
76 FORMAT_SORTIDA  = "vj"         # "vj" (tight crop + resize) /
    "regla-or" (llenç 64×64)
77
78 # Projecció vertical (mètode de segmentació ALTERNATIU,
    opcional)
79 PROJ_SMOOTH_WIN = 9            # finestra de la mitjana mòbil
    del histograma de columnes
80 PROJ_INK_FRAC   = 0.12         # fracció del màxim per
    considerar una columna "amb tinta"
81
82 # -- Fase 3 - OCR -----
    -----
83 CHARS            = list("ABCDEFGHJKLMNOPQRSTUVWXYZ0123456789")
    # 36 classes (A-Z + 0-9)
84 OCR_INPUT_SIZE  = 28
85 CONF_THRESHOLD  = 0.5          # filtre de baixa confiança en
    inferència

```

Listing B.1: alpr/config.py

B.2 Utilitats comunes

```

1  """
2  alpr/common/io.py
3  =====
4  Utilitats bàsiques d'entrada/sortida d'imatges.
5  """
6
7  from pathlib import Path
8

```

```

9 import cv2
10 import numpy as np
11
12 from alpr import config
13
14
15 def load_image(path: Path | str) -> np.ndarray:
16     """
17     Carrega una imatge en format BGR uint8.
18     Llança FileNotFoundError si no es troba o ValueError si no
19     es pot llegir.
20     """
21     path = Path(path)
22     if not path.exists():
23         raise FileNotFoundError(f"Imatge no trobada: '{path}'")
24     img = cv2.imread(str(path))
25     if img is None:
26         raise ValueError(f"No s'ha pogut llegir la imatge:
27         '{path}'")
28     return img
29
30 def save_image(img: np.ndarray, path: Path | str) -> None:
31     """Guarda una imatge BGR. Crea els directoris pares si
32     cal."""
33     path = Path(path)
34     path.parent.mkdir(parents=True, exist_ok=True)
35     cv2.imwrite(str(path), img)
36
37 def to_grayscale(img_bgr: np.ndarray) -> np.ndarray:
38     """Converteix BGR → grisos (1 canal)."""
39     return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
40
41 def iter_images(directory: Path | str):
42     """
43     Generador que retorna (path, img_bgr) per a totes les
44     imatges
45     d'un directori (extensió a config.IMAGE_EXTENSIONS).
46     Les imatges que no es puguin llegir s'ometen amb un avís.
47     """
48     import logging
49     log = logging.getLogger(__name__)
50
51     directory = Path(directory)
52     paths = sorted(
53         p for p in directory.iterdir()
54         if p.suffix.lower() in config.IMAGE_EXTENSIONS
55     )
56     for p in paths:
57         try:
58             yield p, load_image(p)
59         except (FileNotFoundError, ValueError) as e:

```

```
59 log.warning(str(e))
```

Listing B.2: alpr/common/io.py

```
1  """
2  alpr/common/geometry.py
3  =====
4  Utilitats geomètriques transversals: IoU i NMS.
5  """
6
7
8  def iou(a: tuple, b: tuple) -> float:
9      """
10     Intersection over Union de dos bboxes (x, y, w, h).
11     Retorna 0.0 si la unió és zero.
12     """
13     ax, ay, aw, ah = a
14     bx, by, bw, bh = b
15     ix = max(0, min(ax + aw, bx + bw) - max(ax, bx))
16     iy = max(0, min(ay + ah, by + bh) - max(ay, by))
17     inter = ix * iy
18     union = aw * ah + bw * bh - inter
19     return inter / union if union > 0 else 0.0
20
21
22  def nms(
23     boxes: list[tuple],
24     scores: list[float] | None = None,
25     iou_thresh: float = 0.5,
26 ) -> list[tuple]:
27     """
28     Non-Maximum Suppression sobre una llista de bboxes (x, y,
29     w, h).
30
31     Si no es passen scores, s'ordena per àrea descendent (box
32     més gran = prioritat).
33     Retorna la llista de boxes retingudes ordenada per score
34     descendent.
35     """
36     if not boxes:
37         return []
38
39     if scores is None:
40         scores = [b[2] * b[3] for b in boxes] # àrea com a
41         proxy
42
43     # Ordena per score descendent
44     ranked = sorted(zip(scores, boxes), key=lambda x: -x[0])
45     kept: list[tuple] = []
46
47     for _score, box in ranked:
48         if not any(iou(box, k) > iou_thresh for k in kept):
49             kept.append(box)
```

```
47     return kept
```

Listing B.3: alpr/common/geometry.py

```
1  """
2  alpr/common/annotations.py
3  =====
4  Parseig d' anotacions GT i noms de fitxer del projecte.
5
6  Format GT per línia:
7      nom_imatge x y w h MATRICULA (separats per
8      tabulador o espais)
9
10 """
11
12 import logging
13 import re
14 from pathlib import Path
15
16 log = logging.getLogger(__name__)
17
18 # Patrons de nom de fitxer de crop i de caràcter segmentat
19 _RE_CROP = re.compile(r"^(.+)_(:cand|box)(\d+)$")
20 _RE_CHAR = re.compile(r"^(.+)_box(\d+)_char(\d+)\.png$")
21
22 def get_txt_path(img_path: Path) -> Path:
23     """Retorna la ruta del fitxer GT .txt corresponent a
24     img_path."""
25     return img_path.with_suffix(".txt")
26
27 def parse_annotation(line: str) -> dict | None:
28     """
29     Parseja una línia d'anotació GT.
30     Retorna {'filename', 'x', 'y', 'w', 'h', 'plate'} o None
31     si la línia és malformada.
32     """
33     line = line.strip()
34     if not line:
35         return None
36     parts = line.split()
37     if len(parts) < 6:
38         log.warning(f"Línia GT malformada: {line!r}")
39         return None
40     return {
41         "filename": parts[0],
42         "x": int(parts[1]),
43         "y": int(parts[2]),
44         "w": int(parts[3]),
45         "h": int(parts[4]),
46         "plate": parts[5].upper(),
47     }
```

```

48 def load_annotations(gt_path: Path) -> list[dict]:
49     """
50     Carrega totes les anotacions d'un fitxer GT.
51     Retorna llista de dict's (veure parse_annotation).
52     """
53     if not gt_path.exists():
54         return []
55     results = []
56     for line in
57         gt_path.read_text(encoding="utf-8").splitlines():
58         ann = parse_annotation(line)
59         if ann:
60             results.append(ann)
61     return results
62
63 def parse_crop_filename(path: Path) -> tuple[str | None, int |
64     None]:
65     """
66     Extreu (stem_base, idx) de noms '{stem}_cand{n}' o
67     '{stem}_box{n}'.
68     Retorna (None, None) si el nom no coincideix.
69     """
70     m = _RE_CROP.match(path.stem)
71     if m:
72         return m.group(1), int(m.group(2))
73     return None, None
74
75 def parse_char_filename(path: Path) -> tuple[str | None, int |
76     None, int | None]:
77     """
78     Extreu (stem_base, box_idx, char_idx) de noms
79     '{stem}_box{n}_char{i}.png'.
80     Retorna (None, None, None) si no coincideix.
81     """
82     m = _RE_CHAR.match(path.name)
83     if m:
84         return m.group(1), int(m.group(2)), int(m.group(3))
85     return None, None, None

```

Listing B.4: alpr/common/annotations.py

B.3 Fase 1 — Detector

```

1  """
2  alpr/detector/preprocess.py
3  =====
4  Fase 1, pas 1: preprocessament de la imatge per al detector.
5  Grisos → Gaussian → CLAHE.
6  """
7

```

```

8 import cv2
9 import numpy as np
10
11 from alpr import config
12
13
14 def preprocess(img_bgr: np.ndarray) -> np.ndarray:
15     """
16     Converteix a grisos, suavitza amb Gaussian i equalitza el
17     contrast amb CLAHE.
18
19     Retorna imatge 1-canal uint8 lista per a l'extracció de
20     vores.
21     """
22     gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
23     blurred = cv2.GaussianBlur(gray, ksize=config.GAUSS_KSIZE,
24                               sigmaX=config.GAUSS_SIGMA)
25     clahe = cv2.createCLAHE(
26         clipLimit=config.CLAHE_CLIP_LIMIT,
27         tileGridSize=config.CLAHE_TILE_SIZE,
28     )
29     return clahe.apply(blurred)

```

Listing B.5: alpr/detector/preprocess.py

```

1 """
2 alpr/detector/edges.py
3 =====
4 Fase 1, pas 2: Sobel vertical + binarització Otsu.
5
6 Els caràcters d'una matrícula generen una densitat anormalment
7 alta de vores
8 verticals en una regió molt acotada. Explotant aquesta
9 propietat estructural
10 podem localitzar la matrícula sense model entrenat.
11 """
12
13 import cv2
14 import numpy as np
15
16 def sobel_vertical_binary(gray: np.ndarray) ->
17     tuple[np.ndarray, np.ndarray]:
18     """
19     Aplica el filtre de Sobel vertical (dx=1, dy=0, ksize=3) i
20     binaritza
21     amb el mètode d'Otsu.
22
23     Retorna (sobel_abs, binary):
24     - sobel_abs : magnitud de gradient uint8
25     - binary    : imatge binaritzada {0, 255}
26     """
27     sobel_x = cv2.Sobel(gray, ddepth=cv2.CV_16S, dx=1, dy=0,
28                        ksize=3)

```

```

25     sobel_abs = cv2.convertScaleAbs(sobel_x)
26     _, binary = cv2.threshold(
27         sobel_abs, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
28     )
29     return sobel_abs, binary

```

Listing B.6: alpr/detector/edges.py

```

1  """
2  alpr/detector/morphology.py
3  =====
4  Fase 1, pas 3: morfologia matemàtica + etiquetatge de
   components connexos.
5
6  Closing horitzontal: agrupa les vores verticals disperses en
   un blob compacte
7  que correspon a la zona de text de la matrícula.
8  Opening: elimina blobs massa fins (soroll residual).
9  """
10
11 import cv2
12 import numpy as np
13
14 from alpr import config
15
16
17 def morphology_and_label(
18     binary: np.ndarray,
19 ) -> tuple[np.ndarray, int, np.ndarray, np.ndarray]:
20     """
21     Aplica closing horitzontal → opening →
       connectedComponentsWithStats.
22
23     Retorna (morph, num_labels, labels, stats).
24     """
25     close_kernel = cv2.getStructuringElement(cv2.MORPH_RECT,
26         config.CLOSE_KERNEL)
27     closed = cv2.morphologyEx(binary, cv2.MORPH_CLOSE,
28         close_kernel)
29
30     open_kernel = cv2.getStructuringElement(cv2.MORPH_RECT,
31         config.OPEN_KERNEL)
32     morph = cv2.morphologyEx(closed, cv2.MORPH_OPEN,
33         open_kernel)
34
35     num_labels, labels, stats, _ =
36         cv2.connectedComponentsWithStats(
37             morph, connectivity=8
38         )
39     return morph, num_labels, labels, stats

```

Listing B.7: alpr/detector/morphology.py

```

1  """
2  alpr/detector/shape_filter.py
3  =====
4  Fase 1, pas 4: filtre geomètric de components connexos.
5
6  Filosofia de màxim recall: els llindars són deliberadament
7  laxos per no
8  perdre la matrícula real. És preferible tenir falsos positius
9  que perdre
10 la matrícula, ja que el segmentador els descartarà a la fase 2.
11 """
12
13 import numpy as np
14
15 from alpr import config
16
17 def filter_by_shape(
18     num_labels: int,
19     stats: np.ndarray,
20     img_shape: tuple,
21 ) -> list[tuple[int, int, int, int]]:
22     """
23     Filtra els components connexos per àrea relativa, aspect
24     ratio i extent.
25
26     Paràmetres
27     -----
28     num_labels : resultat de connectedComponentsWithStats
29     stats       : array de stats de
30                   cv2.connectedComponentsWithStats
31     img_shape  : (H, W[, C]) de la imatge original
32
33     Retorna
34     -----
35     list[(x, y, w, h)] en píxels de la imatge original.
36     """
37     H, W      = img_shape[:2]
38     img_area = H * W
39     boxes: list[tuple[int, int, int, int]] = []
40
41     for lbl in range(1, num_labels):          # etiqueta 0 =
42         fons                                     fons
43         x, y, w, h, area = stats[lbl]
44
45         if w < config.MIN_WIDTH or h < config.MIN_HEIGHT:
46             continue
47
48         area_ratio = area / img_area
49         if not (config.AREA_RATIO_MIN <= area_ratio <=
50                 config.AREA_RATIO_MAX):
51             continue
52
53         aspect = w / float(h)

```



```

49         if not (config.ASPECT_RATIO_MIN <= aspect <=
50                 config.ASPECT_RATIO_MAX):
51             continue
52
53         extent = area / float(w * h)
54         if extent < config.EXTENT_MIN:
55             continue
56
57         boxes.append((int(x), int(y), int(w), int(h)))
58
59     return boxes

```

Listing B.8: alpr/detector/shape_filter.py

```

1  """
2  alpr/detector/detector.py
3  =====
4  API pública del detector de matrícules (Fase 1).
5
6  Encadena les quatre fases del pipeline morfològic:
7  preprocess → sobel_vertical_binary → morphology_and_label →
8  filter_by_shape
9
10 API pública:
11 detect_boxes(img_bgr) -> list[(x, y, w, h)]
12 detect(img_bgr)       -> list[np.ndarray]          retalls BGR
13 detect_debug(img_bgr) -> dict                      totes les
14                                                         etapes intermèdies
15 """
16
17 import numpy as np
18
19 from .preprocess import preprocess
20 from .edges import sobel_vertical_binary
21 from .morphology import morphology_and_label
22 from .shape_filter import filter_by_shape
23
24 def detect_boxes(img_bgr: np.ndarray) -> list[tuple[int, int,
25 int, int]]:
26     """
27     Detecta bboxes de candidats a matrícula.
28
29     Retorna list[(x, y, w, h)] en píxels de la imatge original.
30     Llista buida si no es detecta res.
31     """
32     enhanced = preprocess(img_bgr)
33     _, binary = sobel_vertical_binary(enhanced)
34     _, num_labels, _, stats = morphology_and_label(binary)
35     return filter_by_shape(num_labels, stats, img_bgr.shape)

```

```

36 def detect(img_bgr: np.ndarray) -> list[np.ndarray]:
37     """
38     Detecta matrícules i retorna els retalls BGR de cada
39     candidat.
40
41     Retorna list[np.ndarray] (BGR uint8). Llista buida si no
42     es detecta res.
43     """
44     boxes = detect_boxes(img_bgr)
45     return [img_bgr[y : y + h, x : x + w] for (x, y, w, h) in
46             boxes]
47
48 def detect_debug(img_bgr: np.ndarray) -> dict:
49     """
50     Com detect_boxes però retorna un dict amb totes les etapes
51     intermèdies,
52     útil per a diagnòstic i visualització.
53
54     Claus del dict:
55     enhanced      : imatge post-CLAHE
56     sobel         : magnitud de gradient Sobel vertical
57     binary        : binaritzada Otsu
58     morph         : post-morfologia
59     boxes         : list[(x, y, w, h)]
60     n_total       : nombre de components connexos (sense el fons)
61     """
62     enhanced = preprocess(img_bgr)
63     sobel_abs, binary = sobel_vertical_binary(enhanced)
64     morph, num_labels, _labels, stats = morphology_and_label(binary)
65     boxes = filter_by_shape(num_labels, stats, img_bgr.shape)
66
67     return {
68         "enhanced": enhanced,
69         "sobel": sobel_abs,
70         "binary": binary,
71         "morph": morph,
72         "boxes": boxes,
73         "n_total": num_labels - 1,
74     }

```

Listing B.9: alpr/detector/detector.py

B.4 Fase 2 — Segmentador

```

1  """
2  alpr/segmenter/normalize.py
3  =====

```

```

4 Fase 2, pas 0 (previ a la detecció): normalització d'escala
   del crop.
5
6 Per què (D9 / ADR-002): les plaques d'aquest dataset són molt
   petites
7 (alçada GT de 18-46 px) i el 'blockSize=31' de la binarització
   adaptativa està
8 calibrat per a una escala concreta. Si es binaritza el crop a
   la seva mida
9 original, el bloc és més gran que el caràcter (o que el crop
   sencer) i la
10 binarització falla → sobre-segmentació. Normalitzant tots els
   crops a una
11 alçada de treball comuna ('NORM_H') abans de detectar, el
   'blockSize' té sempre
12 una escala coherent respecte als caràcters. Com a efecte
   secundari positiu,
13 upscalar caràcters minúsculs millora la qualitat del 28×28
   final.
14 """
15
16 import cv2
17 import numpy as np
18
19 from alpr import config
20
21
22 def normalitza_escala(img: np.ndarray, target_h: int | None =
   None) -> np.ndarray:
23     """
24     Redimensiona la imatge a una alçada de treball comuna
       preservant l'aspect ratio.
25
26     Funciona tant amb imatges BGR (3 canals) com en escala de
       grisos: només
27     afecta les dimensions espacials. Usa INTER_CUBIC per
       ampliar (recupera
28     detall en plaques minúscules) i INTER_AREA per reduir
       (evita aliasing).
29
30     Args:
31         img: imatge d'entrada (BGR uint8 o gris),
              qualsevol mida.
32         target_h: alçada objectiu en px; si és None, s'usa
              config.NORM_H.
33
34     Returns:
35         La imatge redimensionada a target_h d'alçada (mateixa
              que l'entrada si
36         ja hi coincideix).
37     """
38     if target_h is None:
39         target_h = config.NORM_H
40

```

```

41     h, w = img.shape[:2]
42     if h == target_h:
43         return img
44
45     new_w = max(1, int(round(w * target_h / float(h))))
46     interp = cv2.INTER_CUBIC if h < target_h else
47         cv2.INTER_AREA
48     return cv2.resize(img, (new_w, target_h),
49                       interpolation=interp)

```

Listing B.10: alpr/segmenter/normalize.py

```

1  """
2  alpr/segmenter/deskew.py
3  =====
4  Fase 2, pas 1: correcció d'inclinació (deskew) del crop de
5      matrícula.
6
7  Mètode: **regressió lineal sobre els centroides** dels
8      caràcters candidats
9      (ADR-003). Els caràcters d'una matrícula estan alineats
10     horitzontalment per
11     disseny, així que el pendent de la recta que passa pels seus
12     centroides revela
13     directament la inclinació de la placa - un senyal molt més net
14     que la forma
15     global del crop.
16
17     > Per què NO 'minAreaRect' global: es va implementar i
18     descartar (ADR-003). El
19     > núvol global de punts de primer pla (fons binaritzat, vores
20     de la placa,
21     > soroll) forma una forma aproximadament rectangular
22     **alineada amb els eixos**,
23     > de manera que l'angle resultant és gairebé sempre  $\approx 0^\circ$ 
24     independentment de la
25     > inclinació real.
26
27     Esquema de dues passades: aquesta funció fa la **passada 1
28     tolerant** (detecta
29     candidats només per estimar l'angle); la **passada 2
30     estricta** (segmentació
31     final) la fa 'segment()' sobre el crop ja alineat.
32  """
33
34  import cv2
35  import numpy as np
36
37  from alpr import config
38  from .binarize import binarize_adaptive
39  from .contours import extract_contours, filter_geometric
40
41  def estimate_skew_angle(bboxes: list[tuple[int, int, int,

```

```

32     int]]) -> float:
33     """
34     Estima l'angle d'inclinació (graus) per regressió sobre
35     els centroides.
36
37     Ajusta una recta 'cy = m·cx + b' als centroides dels
38     bboxes candidats i
39     retorna 'arctan(m)' en graus. Retorna 0.0 si:
40     - hi ha menys de DESKEW_MIN_CANDIDATS punts (senyal
41     insuficient), o
42     - '|angle| > DESKEW_ANGLE_MAX' (es considera detecció
43     errònia → no rotar).
44     """
45     if len(bboxes) < config.DESKEW_MIN_CANDIDATS:
46         return 0.0
47
48     cx = np.array([x + w / 2.0 for (x, _, w, _) in bboxes])
49     cy = np.array([y + h / 2.0 for (_, y, _, h) in bboxes])
50     slope, _ = np.polyfit(cx, cy, 1)
51     angle = float(np.degrees(np.arctan(slope)))
52
53     if abs(angle) > config.DESKEW_ANGLE_MAX:
54         return 0.0
55     return angle
56
57 def deskew(crop_bgr: np.ndarray) -> tuple[np.ndarray, float]:
58     """
59     Corregeix la inclinació d'un crop de matrícula (passada 1:
60     tolerant).
61
62     Detecta caràcters candidats amb la binarització adaptativa
63     i el filtre
64     geomètric, n'estima l'angle per regressió de centroides i
65     rota el crop al
66     voltant del seu centre. BORDER_REPLICATE evita franges
67     negres als cantons.
68
69     Retorna (aligned_bgr, angle_graus). angle_graus = 0.0 si
70     no s'ha rotat.
71     """
72     thresh = binarize_adaptive(crop_bgr)
73     bboxes = extract_contours(thresh)
74     # Passada 1: filtre TOLERANT (strict=False) relatiu a
75     # l'alçada del crop, només
76     # per obtenir prou candidats per estimar l'angle.
77     candidats = filter_geometric(bboxes, crop_bgr.shape[0],
78     strict=False)
79     angle = estimate_skew_angle(candidats)
80
81     if abs(angle) < 1e-3:
82         return crop_bgr, 0.0
83
84     H_img, W_img = crop_bgr.shape[:2]

```

```

74     M = cv2.getRotationMatrix2D((W_img / 2.0, H_img / 2.0),
75                                   angle, 1.0)
76     aligned = cv2.warpAffine(
77         crop_bgr, M, (W_img, H_img),
78         flags=cv2.INTER_CUBIC,
79         borderMode=cv2.BORDER_REPLICATE,
80     )
81     return aligned, angle

```

Listing B.11: alpr/segmenter/deskew.py

```

1  """
2  alpr/segmenter/binarize.py
3  =====
4  Fase 2, pas 1: binarització adaptativa del crop alineat.
5
6  Usa EXACTAMENT els mateixos paràmetres que el dataset sintètic
7  (02_augment.py)
8  per garantir que la textura de les vores sigui idèntica a la
9  de les imatges
10 reals que el model OCR veurà durant la inferència.
11 """
12
13 import cv2
14 import numpy as np
15
16 from alpr import config
17
18 def binarize_adaptive(aligned_bgr: np.ndarray) -> np.ndarray:
19     """
20     AdaptiveThreshold (MEAN_C, BINARY_INV) + MORPH_CLOSE
21     vertical.
22
23     El CLOSE consolida traços fins ('I') sense fusionar
24     caràcters adjacents
25     gràcies al kernel vertical (1×3).
26
27     Retorna imatge binària uint8: caràcters BLANCS sobre fons
28     NEGRE.
29     """
30     gray = cv2.cvtColor(aligned_bgr, cv2.COLOR_BGR2GRAY)
31     thresh = cv2.adaptiveThreshold(
32         gray, 255,
33         cv2.ADAPTIVE_THRESH_MEAN_C,
34         cv2.THRESH_BINARY_INV,
35         config.ADAPT_BLOCK,
36         config.ADAPT_C,
37     )
38     kernel = cv2.getStructuringElement(cv2.MORPH_RECT,
39                                         config.MORPH_CLOSE_KERNEL)
40     return cv2.morphologyEx(thresh, cv2.MORPH_CLOSE, kernel)

```

Listing B.12: alpr/segmenter/binarize.py

```

1  """
2  alpr/segmenter/contours.py
3  =====
4  Fase 2, pas 2-3: extracció de contorns, filtre geomètric i
    deduplicació IoU.
5  """
6
7  import numpy as np
8  import cv2
9
10 from alpr import config
11 from alpr.common.geometry import iou
12
13
14 def extract_contours(thresh: np.ndarray) -> list[tuple[int,
    int, int, int]]:
15     """
16     Extreu bboxes amb RETR_EXTERNAL.
17
18     RETR_EXTERNAL ignora els forats interiors dels caràcters
        (0, 0, D, A, B...),
19     evitant que el forat es compti com un caràcter separat.
20
21     Retorna list[(x, y, w, h)].
22     """
23     cnts, _ = cv2.findContours(
24         thresh.copy(), cv2.RETR_EXTERNAL,
25         cv2.CHAIN_APPROX_SIMPLE
26     )
27     return [cv2.boundingRect(c) for c in cnts]
28
29 def filter_geometric(
30     bboxes: list[tuple[int, int, int, int]],
31     h_placa: int,
32     strict: bool = True,
33 ) -> list[tuple[int, int, int, int]]:
34     """
35     Filtre geomètric groller RELATIU A L'ALÇADA DE LA PLACA
        ('h_placa'), mai px
36     absoluts. Elimina blobs massa petits/grans o amb aspect
        ratio implausible.
37
38     En mode tolerant ('strict=False', passada 1 per al deskew)
        eixampla els
39     llindars; en mode estricte ('strict=True', passada 2) els
        ajusta. Mateixa
40     lògica que 'filtra_geometric' del segmentador de
        referència.
41
42     - alçada ∈ [H_CHAR_MIN_REL, H_CHAR_MAX_REL]·h_placa
        (×0.7/×1.2 si tolerant)
43     - aspect ratio ∈ [AR_MIN, AR_MAX] (AR_MIN=0.05 preserva la

```

```

44         'I'; ×1.3 si tolerant)
45     - àrea ≥ AREA_MIN_REL·h_placa2 (elimina speckle)
46
47     Retorna la llista filtrada (sense ordenar;
48     remove_overlapping ja ordena per x).
49     """
50     h_min = config.H_CHAR_MIN_REL * (1.0 if strict else
51     0.7) * h_placa
52     h_max = config.H_CHAR_MAX_REL * (1.0 if strict else
53     1.2) * h_placa
54     ar_max = config.AR_MAX * (1.0 if strict else 1.3)
55     area_min = config.AREA_MIN_REL * (h_placa ** 2)
56
57     out: list[tuple[int, int, int, int]] = []
58     for (x, y, w, h) in bboxes:
59         if h <= 0 or w <= 0:
60             continue
61         if not (h_min <= h <= h_max):
62             continue
63         if not (config.AR_MIN <= w / float(h) <= ar_max):
64             continue
65         if (w * h) < area_min:
66             continue
67         out.append((x, y, w, h))
68     return out
69
70 def select_dominant_row(
71     bboxes: list[tuple[int, int, int, int]],
72 ) -> list[tuple[int, int, int, int]]:
73     """
74     De tots els candidats grollers, conserva els que tenen
75     alçada propera a la
76     MEDIANA (la fila dominant de caràcters), robust a alguns
77     blobs de soroll.
78
79     Banda relativa a la mediana: [1-H_MED_TOL,
80     1+H_MED_TOL]·mediana. Equival a
81     'selecciona_fila_per_mediana' del segmentador de
82     referència.
83     """
84     if not bboxes:
85         return []
86     med_h = float(np.median([h for (_, _, _, h) in bboxes]))
87     lo, hi = (1 - config.H_MED_TOL) * med_h, (1 +
88     config.H_MED_TOL) * med_h
89     return [b for b in bboxes if lo <= b[3] <= hi]
90
91 def remove_overlapping(
92     bboxes: list[tuple[int, int, int, int]],
93     iou_thresh: float | None = None,
94 ) -> list[tuple[int, int, int, int]]:
95     """

```



```

89     Deduplicació per IoU: conserva el bbox de més àrea
        (contorn exterior).
90
91     Ordena per àrea descendent perquè el contorn exterior
        sempre entri primer.
92     Retorna la llista retinguda ordenada per x.
93     """
94     if iou_thresh is None:
95         iou_thresh = config.IOU_OVERLAP
96
97     if len(bboxes) <= 1:
98         return list(bboxes)
99
100     kept: list[tuple] = []
101     for bbox in sorted(bboxes, key=lambda b: b[2] * b[3],
102                        reverse=True):
103         if not any(iou(bbox, k) > iou_thresh for k in kept):
104             kept.append(bbox)
105     kept.sort(key=lambda b: b[0])
106     return kept

```

Listing B.13: alpr/segmenter/contours.py

```

1     """
2     alpr/segmenter/projection.py
3     =====
4     Fase 2 - mètode de segmentació ALTERNATIU: projecció vertical.
5
6     Mètode secundari (l'oficial és la detecció de contorns,
        ADR-002). Es conserva per
7     poder **comparar solucions**: sobre el binari de detecció,
        suma els píxels blancs
8     per columna; les bandes de columnes "amb tinta" separades per
        valls són els
9     caràcters. No requereix conèixer el nombre de caràcters a
        priori.
10
11     Mesura de referència: 40 ROIs acceptades (projecció) vs 79
        (contorns) sobre els
12     mateixos 341 crops → els contorns són netament superiors en
        aquest dataset.
13     """
14
15     import numpy as np
16
17     from alpr import config
18
19
20     def _bbox_de_banda(thresh: np.ndarray, x0: int, x1: int):
21         """Donada una banda de columnes [x0, x1), retorna el bbox
            vertical del seu contingut."""
22         banda = thresh[:, x0:x1]
23         files = np.where(banda.sum(axis=1) > 0)[0]
24         if len(files) == 0:

```

```

25         return None
26     y0, y1 = int(files.min()), int(files.max()) + 1
27     return (x0, y0, x1 - x0, y1 - y0)
28
29
30 def detecta_projeccio(thresh: np.ndarray) -> list[tuple[int,
31 int, int, int]]:
32     """
33     Detecta caràcters per projecció vertical sobre el binari
34     (blanc sobre negre).
35
36     Suavitza el histograma de columnes amb una mitjana mòbil,
37     considera "amb tinta"
38     les columnes per sobre d'una fracció del màxim, i extreu
39     un bbox per cada banda
40     continua de columnes amb tinta. Retorna list[(x, y, w, h)].
41     """
42     col = (thresh > 0).sum(axis=0).astype(np.float64)
43     if col.max() <= 0:
44         return []
45
46     kernel = np.ones(config.PROJ_SMOOTH_WIN) /
47         config.PROJ_SMOOTH_WIN
48     col_s = np.convolve(col, kernel, mode="same")
49     llindar = max(config.PROJ_INK_FRAC * col_s.max(), 1.0)
50     amb_tinta = col_s > llindar
51
52     bboxes: list[tuple[int, int, int, int]] = []
53     inici = None
54     for i, val in enumerate(amb_tinta):
55         if val and inici is None:
56             inici = i
57         elif not val and inici is not None:
58             bb = _bbox_de_banda(thresh, inici, i)
59             if bb is not None:
60                 bboxes.append(bb)
61             inici = None
62     if inici is not None:
63         bb = _bbox_de_banda(thresh, inici, thresh.shape[1])
64         if bb is not None:
65             bboxes.append(bb)
66     return bboxes

```

Listing B.14: alpr/segmenter/projection.py

```

1     """
2     alpr/segmenter/validate.py
3     =====
4     Fase 2, pas 4: validació geomètrica del conjunt de caràcters
5     detectats.
6
7     Comprova que els bboxes siguin coherents amb una matrícula
8     real.
9
10    Llindars conservadors: millor rebutjar un crop dubtós que

```

```

    contaminar
8   el dataset d'entrenament amb caràcters espuris.
9   """
10
11  import numpy as np
12
13  from alpr import config
14
15
16  def is_plausible_plate(
17      bboxes: list[tuple[int, int, int, int]],
18      img_w: int,
19  ) -> tuple[bool, str]:
20      """
21      Valida si els bboxes detectats corresponen a una matrícula
22      real.
23
24      Comprovacions:
25      1. Recompte dins [N_CHARS_MIN, N_CHARS_MAX]
26      2. Uniformitat d'alçades:      std_h / h_med <=
27         ROW_STD_H_MAX
28      3. Alineació de baseline:      std_cy / h_med <=
29         ROW_STD_CY_MAX
30      4. Cobertura horitzontal:      (x_max - x_min) / img_w >=
31         WIDTH_OCC_MIN
32
33      Tots els llindars de fila són relatius a la mediana
34      d'alçades o a img_w (mai
35      px absoluts), de manera que no cal l'alçada del crop.
36
37      Retorna (True, '') si vàlid, o (False, motiu) si rebutjat.
38      """
39      n = len(bboxes)
40      if not (config.N_CHARS_MIN <= n <= config.N_CHARS_MAX):
41          return False, f"recompte={n} fora de
42             [{config.N_CHARS_MIN},{config.N_CHARS_MAX}]"
43
44      heights = [h for (_, _, _, h) in bboxes]
45      h_med = float(np.median(heights))
46      h_std = float(np.std(heights))
47      cys = [y + h / 2.0 for (_, y, _, h) in bboxes]
48      cy_std = float(np.std(cys))
49      x_min = min(x for (x, _, _, _) in bboxes)
50      x_max = max(x + w for (x, _, w, _) in bboxes)
51      x_cov = (x_max - x_min) / img_w if img_w > 0 else 0.0
52
53      if h_med > 0 and h_std > config.ROW_STD_H_MAX * h_med:
54          return False, (
55              f"alçades irregulars "
56              f"(std/med={h_std/h_med:.0%}) >
57              {config.ROW_STD_H_MAX:.0%}) "
58          )
59      if h_med > 0 and cy_std > config.ROW_STD_CY_MAX * h_med:
60          return False, (

```

```

54         f"mala alineació vertical "
55         f"(cy_std/med={cy_std/h_med:.0%} >
           {config.ROW_STD_CY_MAX:.0%}) "
56     )
57     if x_cov < config.WIDTH_OCC_MIN:
58         return False, (
59             f"amplada insuficient "
60             f"({x_cov:.0%} < {config.WIDTH_OCC_MIN:.0%}) "
61         )
62     return True, ""

```

Listing B.15: alpr/segmenter/validate.py

```

1  """
2  alpr/segmenter/char_export.py
3  =====
4  Fase 2, pas 5: retall i exportació dels caràcters a 28×28.
5
6  Dos formats seleccionables ('fmt'):
7
8  · "vj" (default) - retall TIGHT del binari de detecció +
   'resize 28×28 INTER_AREA'.
9  Estira l'aspect ratio i omple vora a vora.
10
11 · "regla-or" - replica EXACTAMENT el procés d'entrenament de
   la CNN (la "REGLA
12 D'OR"): glif en GRISOS centrat en un llenç 64×64 amb marge
   (AR preservat, com
13 '01_render_fonts.py') → binarització del "Grup D"
   ('adaptiveThreshold' MEAN_C/
14 INV/31/15 a 64×64, com '02_augment.py') → 'resize 28×28
   INTER_NEAREST'. La
15 binarització es fa a l'escala de 64×64 (no a l'escala de
   detecció), de manera
16 que el gruix de traç i la textura de vores coincideixin amb
   l'entrenament.
17 """
18
19 import cv2
20 import numpy as np
21
22 from alpr import config
23
24
25 def preprocessa_caracter(
26     thresh: np.ndarray,
27     gray: np.ndarray,
28     bbox: tuple[int, int, int, int],
29     fmt: str | None = None,
30 ) -> np.ndarray:
31     """
32     Construeix la imatge 28×28 d'un caràcter (blanc sobre
       negre).
33

```

```

34 Paràmetres
35 -----
36 thresh : binari de detecció (caràcters blancs sobre negre)
37         - font del format "vj".
38 gray   : gris normalitzat i deskew-at (mateix espai de
39         coords que thresh) -
40         font del format "regla-or" (cal el glif en grisos
41         per re-binaritzar).
42 bbox   : (x, y, w, h) del caràcter.
43 fmt    : "vj" (default) o "regla-or".
44
45 Retorna np.ndarray 28×28 uint8 binari (blanc sobre negre).
46 ""
47 if fmt is None:
48     fmt = config.FORMAT_SORTIDA
49
50 x, y, w, h = bbox
51
52 # -- Format "vj": retall tight del binari + resize directe
53 -----
54 if fmt == "vj":
55     crop = thresh[y : y + h, x : x + w]
56     if crop.size == 0:
57         return np.zeros((config.OUTPUT_SIZE,
58                         config.OUTPUT_SIZE), np.uint8)
59     return cv2.resize(
60         crop, (config.OUTPUT_SIZE, config.OUTPUT_SIZE),
61         interpolation=cv2.INTER_AREA,
62     )
63
64 # -- Format "regla-or": REGLA D'OR (render + Grup D
65     exactes sobre grisos) ---
66 if fmt == "regla-or":
67     H, W = gray.shape[:2]
68     # 1px de seguretat per no escapar el contorn, sempre
69     dins de la imatge
70     x0, y0 = max(0, x - 1), max(0, y - 1)
71     x1, y1 = min(W, x + w + 1), min(H, y + h + 1)
72     glif = gray[y0:y1, x0:x1]
73     if glif.size == 0:
74         return np.zeros((config.OUTPUT_SIZE,
75                         config.OUTPUT_SIZE), np.uint8)
76
77     gh, gw = glif.shape[:2]
78     objectiu = config.CANVAS_SIZE - 2 * config.MARGIN
79     # 52 px en la dim. gran
80     escala = objectiu / float(max(gh, gw))
81     nou_w = max(1, int(round(gw * escala)))
82     nou_h = max(1, int(round(gh * escala)))
83     interp = cv2.INTER_AREA if escala < 1.0 else
84         cv2.INTER_CUBIC
85     glif_esc = cv2.resize(glif, (nou_w, nou_h),
86         interpolation=interp)

```

```

77     # Llenç BLANC i enganxat centrat (preserva l'AR; la
78     # dim. petita queda amb marge)
79     llenc = np.full((config.CANVAS_SIZE,
80                     config.CANVAS_SIZE), 255, np.uint8)
81     off_y = (config.CANVAS_SIZE - nou_h) // 2
82     off_x = (config.CANVAS_SIZE - nou_w) // 2
83     llenc[off_y:off_y + nou_h, off_x:off_x + nou_w] =
84         glif_esc
85
86     # Grup D EXACTE (a 64×64) → caràcter BLANC sobre fons
87     # NEGRE
88     binar = cv2.adaptiveThreshold(
89         llenc, 255,
90         cv2.ADAPTIVE_THRESH_MEAN_C,
91         cv2.THRESH_BINARY_INV,
92         config.ADAPT_BLOCK,
93         config.ADAPT_C,
94     )
95     # Reducció final a 28×28 (últim pas, INTER_NEAREST com
96     # l'augmentation)
97     return cv2.resize(
98         binar, (config.OUTPUT_SIZE, config.OUTPUT_SIZE),
99         interpolation=cv2.INTER_NEAREST,
100     )
101
102     raise ValueError(f"Format desconegut: '{fmt}'. Usa 'vj' o
103     'regla-or'.")
104
105 def crops_and_resize(
106     thresh: np.ndarray,
107     gray: np.ndarray,
108     bboxes: list[tuple[int, int, int, int]],
109     fmt: str | None = None,
110 ) -> list[np.ndarray]:
111     """
112     Aplica preprocessa_caracter a tots els bboxes.
113     Retorna llista de np.ndarray 28×28.
114     """
115     return [preprocessa_caracter(thresh, gray, bbox, fmt) for
116             bbox in bboxes]

```

Listing B.16: alpr/segmenter/char_export.py

```

1  """
2  alpr/segmenter/segmenter.py
3  =====
4  API pública del segmentador de caràcters (Fase 2).
5
6  Pipeline per crop:
7      normalitza_escalat → deskew → binarize_adaptive →
8      extract_contours → filter_geometric
9      → select_dominant_row → remove_overlapping →
10     is_plausible_plate → crops_and_resize

```

```

9
10 API pública:
11     segment(roi_bgr)                -> list[np.ndarray]    [] si
12         rebutjat
13     segmenta_caixa(roi_bgr)        -> dict                totes les
14         etapes
15     save_chars(chars, ...)         -> None
16     """
17
18 from pathlib import Path
19
20 import cv2
21 import numpy as np
22
23 from .normalize import normalitza_escalada
24 from .deskew import deskew
25 from .binarize import binarize_adaptive
26 from .contours import (
27     extract_contours, filter_geometric, select_dominant_row,
28     remove_overlapping,
29 )
30 from .projection import detecta_projeccio
31 from .validate import is_plausible_plate
32 from .char_export import crops_and_resize
33
34 from alpr import config
35
36 # =====
37 # API pública
38 # =====
39
40 def segment(
41     roi_bgr: np.ndarray,
42     fmt: str | None = None,
43     metode: str = "contorns",
44 ) -> list[np.ndarray]:
45     """
46     Segmenta els caràcters d'un crop de matrícula.
47
48     Retorna list[np.ndarray 28x28] (blanc sobre negre).
49     Retorna [] si el crop no supera la validació geomètrica
50     (no és matrícula).
51
52     El pipeline retorna [] per a falsos positius del detector:
53     és el mecanisme
54     de rebuig de la Fase 2 (veure contractes a config.py /
55     guia §4).
56
57     'metode': "contorns" (oficial, ADR-002) o "projeccio"
58     (alternatiu, per comparar).
59     """

```

```

54 # Normalitza l'escala abans de detectar (D9/ADR-002):
    # imprescindible perquè
55 # blockSize=31 sigui coherent amb caràcters de plaques de
    # 18-46 px. H i W es
56 # prenen de la imatge normalitzada perquè els bboxes (i la
    # cobertura
57 # d'is_plausible_plate) quedin al mateix espai de
    # coordenades.
58 roi_bgr = normalitza_escala(roi_bgr)
59 H, W = roi_bgr.shape[:2]
60
61 aligned, _angle = deskew(roi_bgr)
62 thresh = binarize_adaptive(aligned)
63 all_bboxes = (detecta_projeccio(thresh) if metode
    == "projeccio"
64                  else extract_contours(thresh))
65 filtered = filter_geometric(all_bboxes, H)
66 filtered = select_dominant_row(filtered)
67 filtered = remove_overlapping(filtered)
68 accepted, _reason = is_plausible_plate(filtered, W)
69
70 if not accepted:
71     return []
72
73 # El format "regla-or" re-binaritza el glif en grisos
    # (Grup D); el "vj" usa
74 # el binari. Passem tots dos: el gray és l'aligned
    # normalitzat i deskew-at.
75 gray = cv2.cvtColor(aligned, cv2.COLOR_BGR2GRAY)
76 return crops_and_resize(thresh, gray, filtered, fmt)
77
78
79 def segmenta_caixa(
80     roi_bgr: np.ndarray,
81     fmt: str | None = None,
82     metode: str = "contorns",
83 ) -> dict:
84     """
85     Com segment però retorna un dict complet amb totes les
        etapes.
86     Útil per a diagnòstic i visualització.
87
88     Claus del dict:
89         aligned, angle, thresh,
90         all_bboxes, filtered_bboxes,
91         accepted, rejection_reason,
92         chars (list[np.ndarray 28x28] o [])
93     """
94     # Normalització d'escala prèvia (D9/ADR-002), idèntica a
        # segment().
95     roi_bgr = normalitza_escala(roi_bgr)
96     H, W = roi_bgr.shape[:2]
97
98     aligned, angle = deskew(roi_bgr)

```



```

99     thresh                = binarize_adaptive(aligned)
100     all_bboxes            = (detecta_projeccio(thresh) if metode
    == "projeccio"
101                             else extract_contours(thresh))
102     filtered               = filter_geometric(all_bboxes, H)
103     filtered               = select_dominant_row(filtered)
104     filtered               = remove_overlapping(filtered)
105     accepted, reason      = is_plausible_plate(filtered, W)
106     gray                   = cv2.cvtColor(aligned,
    cv2.COLOR_BGR2GRAY)
107     chars                  = crops_and_resize(thresh, gray,
    filtered, fmt) if accepted else []
108
109     return {
110         "aligned":         aligned,
111         "angle":           angle,
112         "thresh":          thresh,
113         "all_bboxes":      all_bboxes,
114         "filtered_bboxes": filtered,
115         "accepted":        accepted,
116         "rejection_reason": reason,
117         "chars":           chars,
118     }
119
120
121 def save_chars(
122     chars: list[np.ndarray],
123     roi_name: str,
124     out_dir: Path | str,
125     metadata: dict | None = None,
126 ) -> None:
127     """
128     Guarda els caràcters 28×28 a disc.
129
130     Nomenclatura: {roi_name}_char{i:02d}.png
131     Si metadata conté 'gt' (string), s'afegeix al nom com a
    referència.
132
133     Crea out_dir si no existeix.
134     """
135     out_dir = Path(out_dir)
136     out_dir.mkdir(parents=True, exist_ok=True)
137
138     for i, char_img in enumerate(chars):
139         fname = f"{roi_name}_char{i:02d}.png"
140         cv2.imwrite(str(out_dir / fname), char_img)

```

Listing B.17: alpr/segmenter/segmenter.py

B.5 Fase 3 — OCR: generació de dades

```

1  """

```

```

2 01_render_fonts.py
3 =====
4 Pas 2 del pla d'entrenament OCR.
5
6 Recorre totes les fonts .ttf / .otf de resources/fonts/,
   renderitza cada un
7 dels 62 caràcters alfanumèrics (A-Z, 0-9) i guarda les imatges
   base a:
8
9     data/synthetic/raw/{classe}/{font_stem}_{classe}.png
10
11 Cada imatge és en escala de grisos, 64×64 px, negre sobre
   blanc.
12 No s'aplica cap augmentation aquí - les imatges base han de
   ser netes.
13 L'augmentation es fa al pas següent (02_augment.py).
14
15 Ús
16 --
17     python 01_render_fonts.py
18     python 01_render_fonts.py --fonts_dir resources/fonts
       --out_dir data/synthetic/raw --size 64
19 """
20
21 import argparse
22 import logging
23 from pathlib import Path
24
25 from PIL import Image, ImageDraw, ImageFont
26
27
28 # --- Constants -----
   -----
29
30 # Tots els caràcters que pot contenir una matrícula
31 CHARS = list("ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789") # 62
   classes
32
33 # Mida del llenç de sortida (en píxels)
34 CANVAS_SIZE = 64
35
36 # Marge interior: el caràcter ocupa com a màxim CANVAS_SIZE -
   2*MARGIN px
37 MARGIN = 6
38
39 # --- Logger -----
   -----
40
41 logging.basicConfig(
42     level=logging.INFO,
43     format="%(levelname)s %(message)s"
44 )
45 log = logging.getLogger(__name__)
46

```

```

47
48 # =====
49 # Funcions principals
50 # =====
51
52 def find_fonts(fonts_dir: Path) -> list[Path]:
53     """
54     Retorna la llista de tots els fitxers .ttf i .otf que hi
55     ha a fonts_dir,
56     recursivament (per si les fonts estan en subcarpetes).
57     """
58     ttf = list(fonts_dir.rglob("*.ttf"))
59     otf = list(fonts_dir.rglob("*.otf"))
60     fonts = sorted(ttf + otf)
61     if not fonts:
62         raise FileNotFoundError(
63             f"No s'han trobat fonts .ttf/.otf a '{fonts_dir}'.",
64             "Comprova que el directori és correcte."
65         )
66     return fonts
67
68 def fit_font(font_path: Path, char: str, target_size: int) ->
69     ImageFont.FreeTypeFont:
70     """
71     Tria la mida de font (en punts) perquè el caràcter encaixi
72     dins
73     target_size × target_size px amb un marge MARGIN a cada
74     costat.
75
76     La cerca és binària: prova mides creixents fins que el
77     caràcter supera
78     l'àrea disponible, i retorna la mida anterior.
79
80     Retorna un objecte ImageFont llest per usar amb ImageDraw.
81     """
82     max_dim = target_size - 2 * MARGIN # amplada i alçada
83     màximes del caràcter
84     font_size = 8
85     best_font = ImageFont.truetype(str(font_path),
86                                     size=font_size)
87
88     for font_size in range(8, 200):
89         font = ImageFont.truetype(str(font_path),
90                                     size=font_size)
91         # getbbox retorna (left, top, right, bottom) del
92         caràcter
93         bbox = font.getbbox(char)
94         char_w = bbox[2] - bbox[0]
95         char_h = bbox[3] - bbox[1]
96         if char_w > max_dim or char_h > max_dim:

```

```

89         break
90         best_font = font
91
92     return best_font
93
94
95 def render_char(char: str, font: ImageFont.FreeTypeFont,
96               canvas_size: int) -> Image.Image:
97     """
98     Renderitza un únic caràcter centrat en un llenç blanc de
99     canvas_size×canvas_size.
100
101     Convenció de color:
102     - Fons : blanc (255)
103     - Caràcter: negre (0)
104
105     El segmentador lliura imatges binaritzades amb caràcters
106     BLANCS sobre fons NEGRE.
107     Aquí ho invertim deliberadament: treballem en l'espai
108     natural (tinta fosca sobre
109     paper blanc) i deixem la inversió per al pas
110     d'augmentation + binarització,
111     on s'aplicarà el mateix adaptiveThreshold que el
112     segmentador real.
113     """
114     img = Image.new("L", (canvas_size, canvas_size),
115                   color=255) # fons blanc
116     draw = ImageDraw.Draw(img)
117
118     # Calculem la posició centrada
119     bbox = font.getbbox(char)
120     char_w = bbox[2] - bbox[0]
121     char_h = bbox[3] - bbox[1]
122     x = (canvas_size - char_w) // 2 - bbox[0]
123     y = (canvas_size - char_h) // 2 - bbox[1]
124
125     draw.text((x, y), char, font=font, fill=0) # text negre
126     return img
127
128
129 def render_all(fonts_dir: Path, out_dir: Path, canvas_size:
130 int) -> None:
131     """
132     Pipeline complet: per cada font i per cada caràcter,
133     renderitza i guarda.
134
135     Estructura de sortida:
136     out_dir/
137         A/ font1_A.png font2_A.png ...
138         B/ ...
139         O/ ...
140     """
141     fonts = find_fonts(fonts_dir)
142     log.info(f"Fonts trobades : {len(fonts)}")

```

```

135     log.info(f"Caràcters per font : {len(CHARS)}")
136     log.info(f"Total imatges base : {len(fonts) * len(CHARS)}")
137     log.info(f"Directorí sortida : {out_dir.resolve()}")
138     log.info("-" * 50)
139
140     # Crea les carpetes de classe si no existeixen
141     for char in CHARs:
142         # Els dígit i lletres van directament com a nom de
143         carpeta.
144         # No hi ha conflicte en cap SO perquè cap nom és
145         reservat.
146         (out_dir / char).mkdir(parents=True, exist_ok=True)
147
148     n_ok = 0
149     n_skip = 0
150
151     for font_path in fonts:
152         font_stem = font_path.stem # nom del fitxer sense
153         extensió
154
155         for char in CHARs:
156             out_path = out_dir / char /
157             f"{font_stem}_{char}.png"
158
159             # Salta si ja existeix (permet reprendre una
160             execució interrompuda)
161             if out_path.exists():
162                 n_skip += 1
163                 continue
164
165             try:
166                 font = fit_font(font_path, char, canvas_size)
167                 img = render_char(char, font, canvas_size)
168                 img.save(str(out_path))
169                 n_ok += 1
170             except Exception as e:
171                 log.warning(f" ERROR {font_stem} / '{char}':
172                             {e}")
173                 n_skip += 1
174
175             log.info(f" OK {font_stem}")
176
177     log.info("-" * 50)
178     log.info(f"Imatges generades : {n_ok}")
179     log.info(f"Saltades / error : {n_skip}")
180     log.info(f"Resultat a : {out_dir.resolve()}")
181
182     # =====
183     =====
184     # Verificació visual (opcional)
185     # =====
186     =====

```

```

181 def show_sample(out_dir: Path, n_chars: int = 10, n_fonts: int
182               = 3) -> None:
183     """
184     Mostra una graella de mostra: n_chars columnes x n_fonts
185     files.
186     Útil per verificar visualment que el renderitzat és
187     correcte.
188     Requereix matplotlib.
189     """
190     try:
191         import matplotlib.pyplot as plt
192         import random
193     except ImportError:
194         log.warning("matplotlib no disponible. Salta la
195                     verificació visual.")
196         return
197
198     sample_chars = CHARS[:n_chars]
199     fig, axes = plt.subplots(n_fonts, n_chars,
200                             figsize=(n_chars * 1.2, n_fonts * 1.5))
201
202     for row, char in enumerate(sample_chars):
203         char_dir = out_dir / char
204         imgs = sorted(char_dir.glob("*.png"))
205         if not imgs:
206             continue
207         sample_fonts = random.sample(imgs, min(n_fonts,
208                                                  len(imgs)))
209         for col, img_path in enumerate(sample_fonts):
210             ax = axes[col][row] if n_fonts > 1 else axes[row]
211             ax.imshow(Image.open(img_path), cmap="gray",
212                        vmin=0, vmax=255)
213             ax.set_title(char, fontsize=8)
214             ax.axis("off")
215
216     plt.suptitle("Mostra d'imatges base renderitzades",
217                 fontsize=11)
218     plt.tight_layout()
219     plt.show()
220
221 # =====
222 # Punt d'entrada
223 # =====
224
225 def main() -> None:
226     parser = argparse.ArgumentParser(
227         description="Renderitza caràcters alfanumèrics des de
228                     fonts .ttf/.otf."
229     )
230     parser.add_argument(
231         "--fonts_dir", default="resources/fonts",

```

```

224         help="Directori amb les fonts .ttf/.otf (default:
225             resources/fonts)"
226     )
227     parser.add_argument(
228         "--out_dir", default="data/synthetic/raw",
229         help="Directori de sortida (default:
230             data/synthetic/raw)"
231     )
232     parser.add_argument(
233         "--size", type=int, default=CANVAS_SIZE,
234         help=f"Mida del llenç en píxels (default:
235             {CANVAS_SIZE})"
236     )
237     parser.add_argument(
238         "--preview", action="store_true",
239         help="Mostra una graella visual de mostra al final"
240     )
241     args = parser.parse_args()
242
243     render_all(
244         fonts_dir=Path(args.fonts_dir),
245         out_dir=Path(args.out_dir),
246         canvas_size=args.size,
247     )
248
249     if args.preview:
250         show_sample(Path(args.out_dir))
251
252 if __name__ == "__main__":
253     main()

```

Listing B.18: 01_render_fonts.py

```

1  """
2  02_augment.py
3  =====
4  Pas 4 del pla d'entrenament OCR.
5
6  Llegeix les imatges base de data/synthetic/raw/{classe}/*.png
7  (64×64 px,
8  escala de grisos, caràcter NEGRE sobre fons BLANC) i genera un
9  dataset
10 augmentat estructurat per a entrenament:
11
12     data/synthetic/
13     +-- train/{classe}/*.png    (80%)
14     +-- val/{classe}/*.png      (20%)
15
16 Cada imatge de sortida: 28×28 px, escala de grisos, binària
17 (0/255),
18 caràcter BLANC sobre fons NEGRE (convenció del segmentador
19 real).

```

```

17  Ús
18  --
19      python 02_augment.py
20      python 02_augment.py --samples_per_class 500 --val_split
        0.2 --seed 42
21      python 02_augment.py --samples_per_class 20 --preview
22  """
23
24  import argparse
25  import json
26  import logging
27  import random
28  from pathlib import Path
29
30  import cv2
31  import numpy as np
32
33
34  # --- Constants: probabilitats i rangs d'augmentació -----
        -----
35
36  # Grup A - Degradació geomètrica
37  PROB_ROTATION    = 0.7
38  PROB_SHEAR       = 0.4
39  PROB_TRANSLATE   = 0.5
40  PROB_SCALE       = 0.5
41  ROT_MAX_DEG      = 8.0          #  $\pm 8^\circ$ 
42  SHEAR_MAX        = 0.1          #  $\pm 0.1$ 
43  TRANSLATE_MAX    = 0.10         #  $\pm 10\%$  de la mida del llenç
44  SCALE_RANGE      = (0.85, 1.15)
45
46  # Grup B - Degradació de traç
47  PROB_EROSION     = 0.3          # erosió de tinta → traç més prim
48  PROB_DILATION    = 0.3          # dilatació de tinta → traç més
        gruixut
49  PROB_NOISE       = 0.4
50  NOISE_MAX_PCT    = 0.03         # fins a 3% de píxels afectats per
        soroll
51
52  # Grup C - Degradació de contorn
53  PROB_CROP_SIDE   = 0.3
54  PROB_OFFSET      = 0.4
55  CROP_MAX_PX      = 3            # 0-3 px tallats d'un costat
56  OFFSET_MAX_PX    = 3            #  $\pm 3$  px descentrat del caràcter
57
58  # Mides d'imatge
59  INPUT_SIZE       = 64           # mida de les imatges base
60  OUTPUT_SIZE      = 28           # mida de sortida (igual que el
        segmentador)
61
62  # --- Logger -----
        -----
63
64  logging.basicConfig(level=logging.INFO, format="%(levelname)s

```



```

65     %(message)s")
66 log = logging.getLogger(__name__)
67
68 # =====
69 # Grup A - Degradació geomètrica
70 # =====
71
72 def apply_geometric(img: np.ndarray) -> np.ndarray:
73     """
74     Combina rotació, shear, desplaçament i escala en una sola
75     matriu afí.
76
77     borderValue=255: les zones buides s'omplen amb BLANC
78     (fons), perquè en
79     aquesta fase el caràcter és negre sobre blanc; no volem
80     omplir de tinta.
81     """
82     h, w = img.shape[:2]
83     cx, cy = w / 2.0, h / 2.0
84     M = np.eye(3, dtype=np.float64)
85
86     if random.random() < PROB_SCALE:
87         s = random.uniform(*SCALE_RANGE)
88         Ms = np.float64([[s, 0, cx * (1 - s)],
89                         [0, s, cy * (1 - s)],
90                         [0, 0, 1]])
91         M = Ms @ M
92
93     if random.random() < PROB_ROTATION:
94         angle = random.uniform(-ROT_MAX_DEG, ROT_MAX_DEG)
95         rot_2x3 = cv2.getRotationMatrix2D((cx, cy), angle, 1.0)
96         Mr = np.vstack([rot_2x3, [0, 0, 1]])
97         M = Mr @ M
98
99     if random.random() < PROB_SHEAR:
100         shear = random.uniform(-SHEAR_MAX, SHEAR_MAX)
101         # Shear centrat en cy per minimitzar el desplaçament
102         # vertical
103         Msh = np.float64([[1, shear, -cy * shear],
104                         [0, 1, 0],
105                         [0, 0, 1]])
106         M = Msh @ M
107
108     if random.random() < PROB_TRANSLATE:
109         tx = random.uniform(-TRANSLATE_MAX, TRANSLATE_MAX) * w
110         ty = random.uniform(-TRANSLATE_MAX, TRANSLATE_MAX) * h
111         Mt = np.float64([[1, 0, tx],
112                         [0, 1, ty],
113                         [0, 0, 1]])
114         M = Mt @ M

```

```

112     out = cv2.warpAffine(img, M[:2], (w, h),
113                           flags=cv2.INTER_LINEAR,
114                           borderMode=cv2.BORDER_CONSTANT,
115                           borderValue=255)
116     return out
117
118
119 # =====
120 # Grup B - Degradació de traç
121 # =====
122
123 def apply_stroke(img: np.ndarray) -> np.ndarray:
124     """
125     CONVENCIÓ OPENCV (important):
126     Treballem en l'espai "caràcter negre (0) sobre fons
127         blanc (255)".
128     cv2.erode → calcula el mínim → el negre s'expandeix →
129         traç MÉS GRUIXUT.
130     cv2.dilate → calcula el màxim → el blanc s'expandeix →
131         traç MÉS PRIM.
132     Per tant:
133     · "erosió de tinta" (traç prim) → cv2.dilate
134     · "dilatació de tinta" (traç gruixut) → cv2.erode
135     """
136     img = img.copy()
137
138     # Erosió i dilatació són mútuament excloents en la mateixa
139     # imatge
140     r_e = random.random()
141     r_d = random.random()
142
143     if r_e < PROB_EROSION:
144         # Traç més prim: dilata el blanc (màxim del veïnat)
145         ksize = random.choice([2, 3])
146         kernel = np.ones((ksize, ksize), np.uint8)
147         img = cv2.dilate(img, kernel, iterations=1)
148     elif r_d < PROB_DILATION:
149         # Traç més gruixut: eroda el blanc, expandeix el negre
150         # (mínim del veïnat)
151         ksize = random.choice([2, 3])
152         kernel = np.ones((ksize, ksize), np.uint8)
153         img = cv2.erode(img, kernel, iterations=1)
154
155     if random.random() < PROB_NOISE:
156         noise_pct = random.uniform(0.01, NOISE_MAX_PCT)
157         n_px = max(1, int(noise_pct * img.size))
158         # Sal: píxels blancs aleatoris
159         rows = np.random.randint(0, img.shape[0], n_px // 2)
160         cols = np.random.randint(0, img.shape[1], n_px // 2)
161         img[rows, cols] = 255
162         # Pebre: píxels negres aleatoris
163         rows = np.random.randint(0, img.shape[0], n_px // 2)

```

```

159         cols = np.random.randint(0, img.shape[1], n_px // 2)
160         img[rows, cols] = 0
161
162     return img
163
164
165 # =====
166 # Grup C - Degradació de contorn
167 # =====
168
169 def apply_contour(img: np.ndarray) -> np.ndarray:
170     """
171     Imita el tall de bounding box del segmentador i el
172     descentrat del caràcter.
173     Les zones descobertes s'omplen amb blanc (fons).
174     """
175     h, w = img.shape[:2]
176     out = img.copy()
177
178     if random.random() < PROB_CROP_SIDE:
179         px = random.randint(1, CROP_MAX_PX)
180         side = random.choice(["top", "bottom", "left",
181                               "right"])
182         if side == "top":
183             out[:px, :] = 255
184         elif side == "bottom":
185             out[h - px:, :] = 255
186         elif side == "left":
187             out[:, :px] = 255
188         else:
189             out[:, w - px:] = 255
190
191     if random.random() < PROB_OFFSET:
192         dx = random.randint(-OFFSET_MAX_PX, OFFSET_MAX_PX)
193         dy = random.randint(-OFFSET_MAX_PX, OFFSET_MAX_PX)
194         Mt = np.float64([[1, 0, dx],
195                          [0, 1, dy]])
196         out = cv2.warpAffine(out, Mt, (w, h),
197                               borderMode=cv2.BORDER_CONSTANT,
198                               borderValue=255)
199
200     return out
201
202 # =====
203 # Grup D - Binarització final (sempre s'aplica)
204 # =====
205
206 def apply_binarize(img: np.ndarray) -> np.ndarray:
207     """

```

```

207     Pas crític: aplica EXACTAMENT els mateixos paràmetres que
208         el segmentador real.
209
210     THRESH_BINARY_INV: píxels per sota del llindar → 255, per
211         sobre → 0.
212     Com que el caràcter és negre (per sota del llindar blanc
213         del fons),
214     queda invertit: caràcter BLANC sobre fons NEGRE.
215     Això garanteix que la textura de les vores sigui idèntica
216         a la de les
217     imatges reals que el model veurà durant la inferència.
218     """
219     return cv2.adaptiveThreshold(
220         img, 255,
221         cv2.ADAPTIVE_THRESH_MEAN_C,
222         cv2.THRESH_BINARY_INV,
223         blockSize=31,
224         C=15,
225     )
226
227 # =====
228 # Pipeline d'augmentation
229 # =====
230
231 def augment(img: np.ndarray) -> np.ndarray:
232     """
233     Orquestra els quatre grups en ordre estricte:
234         A (geometria) → B (traç) → C (contorn) → D (binarització)
235
236     Entrada : 64×64, grisos, caràcter NEGRE sobre fons BLANC.
237     Sortida : 28×28, binària (0/255), caràcter BLANC sobre
238         fons NEGRE.
239
240     Tot el processament es fa a 64×64; la reducció a 28×28 és
241         l'últim pas
242     per evitar que les transformacions geomètriques tallin el
243         caràcter.
244     """
245     img = apply_geometric(img)
246     img = apply_stroke(img)
247     img = apply_contour(img)
248     img = apply_binarize(img)
249     img = cv2.resize(img, (OUTPUT_SIZE, OUTPUT_SIZE),
250                       interpolation=cv2.INTER_NEAREST)
251     return img
252
253 # =====
254 # Construcció del dataset
255 # =====

```

```

=====
251
252 def build_dataset(raw_dir: Path, out_dir: Path,
253                   samples_per_class: int, val_split: float) ->
254                   None:
255     """
256     Recorre les classes, genera les mostres augmentades, fa el
257     split
258     train/val i guarda les imatges i el fitxer
259     class_to_idx.json.
260
261     Si una classe té menys imatges base que samples_per_class,
262     les bases
263     es reutilitzen cíclicament amb augmentacions diverses.
264     """
265     class_dirs = sorted([d for d in raw_dir.iterdir() if
266                          d.is_dir()])
267     if not class_dirs:
268         raise FileNotFoundError(f"No s'han trobat carpetes de
269                                classe a '{raw_dir}'")
270
271     classes = [d.name for d in class_dirs]
272     class_to_idx = {cls: idx for idx, cls in
273                     enumerate(classes)}
274
275     n_val = max(1, int(samples_per_class * val_split))
276     n_train = samples_per_class - n_val
277
278     log.info(f"Classes trobades : {len(classes)}")
279     log.info(f"Mostres per classe : {samples_per_class}
280              (train={n_train}, val={n_val})")
281     log.info(f"Divisió validació : {val_split:.0%}")
282     log.info(f"Directorí sortida : {out_dir.resolve()}")
283     log.info("-" * 50)
284
285     total_ok = 0
286     for cls in classes:
287         base_imgs = sorted((raw_dir / cls).glob("*.png"))
288         if not base_imgs:
289             log.warning(f" SALT '{cls}': no hi ha imatges
290                          base")
291             continue
292
293         (out_dir / "train" / cls).mkdir(parents=True,
294                                          exist_ok=True)
295         (out_dir / "val" / cls).mkdir(parents=True,
296                                       exist_ok=True)
297
298         for split, n_samples in [("train", n_train), ("val",
299                                                       n_val)]:
300             for i in range(n_samples):
301                 # Distribució cíclica: si hi ha menys bases
302                 # que mostres,
303                 # se'n generen múltiples variants de cada base

```

```

291         base_path = base_imgs[i % len(base_imgs)]
292         base_img = cv2.imread(str(base_path),
293                               cv2.IMREAD_GRAYSCALE)
294         if base_img is None:
295             log.warning(f" ERROR llegend
296                           {base_path.name}")
297             continue
298
299         aug_img = augment(base_img)
300         out_path = out_dir / split / cls /
301                     f"img_{i:05d}.png"
302         cv2.imwrite(str(out_path), aug_img)
303         total_ok += 1
304
305         log.info(f" OK '{cls}' ({len(base_imgs)} bases →
306                   {n_train} train + {n_val} val)")
307
308     idx_path = out_dir / "class_to_idx.json"
309     idx_path.write_text(json.dumps(class_to_idx, indent=2,
310                                   ensure_ascii=False))
311
312     log.info("-" * 50)
313     log.info(f"Imatges generades : {total_ok}")
314     log.info(f"class_to_idx      : {idx_path}")
315
316 # =====
317 # Preview visual (opcional, --preview)
318 # =====
319
320 def preview(raw_dir: Path, n_chars: int = 8, n_variants: int =
321 5) -> None:
322     """
323     Graella de verificació: original (redimensionat) | N
324     variants augmentades.
325     Cada fila és una classe; la primera columna és la imatge
326     base de referència.
327     """
328     try:
329         import matplotlib.pyplot as plt
330     except ImportError:
331         log.warning("matplotlib no disponible. Salta el
332                     preview.")
333         return
334
335     class_dirs = sorted([d for d in raw_dir.iterdir() if
336                          d.is_dir()])
337     sample_classes = class_dirs[:n_chars]
338     n_cols = 1 + n_variants
339
340     fig, axes = plt.subplots(len(sample_classes), n_cols,
341                              figsize=(n_cols * 1.5,

```

```

len(sample_classes) * 1.5))
333
334 for row, cls_dir in enumerate(sample_classes):
335     base_imgs = sorted(cls_dir.glob("*.png"))
336     if not base_imgs:
337         continue
338     base_img = cv2.imread(str(base_imgs[0]),
339                             cv2.IMREAD_GRAYSCALE)
340
341     ax_row = axes[row] if len(sample_classes) > 1 else axes
342
343     # Primera columna: original redimensionat per
344     comparació visual
345     orig_small = cv2.resize(base_img, (OUTPUT_SIZE,
346                                     OUTPUT_SIZE),
347                             interpolation=cv2.INTER_AREA)
348     ax_row[0].imshow(orig_small, cmap="gray", vmin=0,
349                      vmax=255)
350     if row == 0:
351         ax_row[0].set_title("original", fontsize=8)
352         ax_row[0].set_ylabel(cls_dir.name, fontsize=9,
353                             rotation=0, labelpad=14)
354         ax_row[0].axis("off")
355
356     # Columnes restants: variants augmentades
357     for col in range(n_variants):
358         aug_img = augment(base_img.copy())
359         ax_row[col + 1].imshow(aug_img, cmap="gray",
360                                vmin=0, vmax=255)
361         if row == 0:
362             ax_row[col + 1].set_title(f"aug {col + 1}",
363                                       fontsize=8)
364             ax_row[col + 1].axis("off")
365
366     plt.suptitle("Preview augmentation (original |
367                  variants)", fontsize=11)
368     plt.tight_layout()
369     plt.show()
370
371 # =====
372 # Punt d'entrada
373 # =====
374
375 def main() -> None:
376     parser = argparse.ArgumentParser(
377         description="Genera el dataset augmentat per a
378                     l'entrenament OCR."
379     )
380     parser.add_argument(
381         "--raw_dir", default="data/synthetic/raw",
382         help="Directorio amb les imatges base (default:

```

```

375         data/synthetic/raw)",
376     )
377     parser.add_argument(
378         "--out_dir", default="data/synthetic",
379         help="Directori de sortida (default: data/synthetic)",
380     )
381     parser.add_argument(
382         "--samples_per_class", type=int, default=500,
383         help="Nombre total de mostres per classe (default: 500)",
384     )
385     parser.add_argument(
386         "--val_split", type=float, default=0.2,
387         help="Proporció per a validació (default: 0.2)",
388     )
389     parser.add_argument(
390         "--seed", type=int, default=42,
391         help="Llavor aleatòria per a reproductibilitat (default: 42)",
392     )
393     parser.add_argument(
394         "--preview", action="store_true",
395         help="Mostra la graella visual d'augmentations i surt (no genera dataset)",
396     )
397     args = parser.parse_args()
398
399     random.seed(args.seed)
400     np.random.seed(args.seed)
401
402     raw_dir = Path(args.raw_dir)
403     out_dir = Path(args.out_dir)
404
405     if not 0.0 < args.val_split < 1.0:
406         raise ValueError(f"--val_split ha de ser entre 0 i 1, rebut: {args.val_split}")
407
408     if args.preview:
409         preview(raw_dir)
410     else:
411         build_dataset(raw_dir, out_dir,
412                       args.samples_per_class, args.val_split)
413
414 if __name__ == "__main__":
415     main()

```

Listing B.19: 02_augment.py

B.6 Fase 3 — OCR: model, entrenament i avaluació


```

1  """
2  alpr/ocr/model.py
3  =====
4  Arquitectura CharCNN per a classificació de caràcters 1×28×28.
5
6  Separat de train.py perquè infer.py i eval_real.py el puguin
   importar
7  directament sense haver de fer importlib sobre un script amb
   nom numèric.
8
9  Flux de dimensions:
10     1×28×28
11     → Conv(1→32, k=3, p=1) + BN + ReLU + MaxPool(2) → 32×14×14
12     → Conv(32→64, k=3, p=1) + BN + ReLU + MaxPool(2) → 64×7×7
13     → Dropout(0.4)
14     → Flatten → 3136
15     → Linear(3136→256) + ReLU
16     → Dropout(0.3)
17     → Linear(256→n_classes) [logits, sense Softmax]
18  """
19
20  import torch
21  import torch.nn as nn
22
23
24  class CharCNN(nn.Module):
25      """
26      CNN compacte per a la classificació de caràcters
        alfanumèrics (36 classes: A-Z, 0-9).
27      Entrada: tensor 1×28×28 normalitzat a [-1, 1].
28      Sortida: logits crus (sense Softmax) de mida n_classes.
29      """
30
31      def __init__(self, n_classes: int) -> None:
32          super().__init__()
33
34          self.features = nn.Sequential(
35              # Bloc 1 - detecta contorns i traços bàsics
36              nn.Conv2d(1, 32, kernel_size=3, padding=1),
37              nn.BatchNorm2d(32),
38              nn.ReLU(inplace=True),
39              nn.MaxPool2d(2), # 28 → 14
40
41              # Bloc 2 - combina característiques de nivell mig
42              nn.Conv2d(32, 64, kernel_size=3, padding=1),
43              nn.BatchNorm2d(64),
44              nn.ReLU(inplace=True),
45              nn.MaxPool2d(2), # 14 → 7
46
47              nn.Dropout(0.4),
48          )
49
50          self.classifier = nn.Sequential(

```

```

51         nn.Flatten(),
52         nn.Linear(64 * 7 * 7, 256),
53         nn.ReLU(inplace=True),
54         nn.Dropout(0.3),
55         nn.Linear(256, n_classes),          #
56         CrossEntropyLoss ja inclou Softmax
57     )
58
59     def forward(self, x: torch.Tensor) -> torch.Tensor:
60         return self.classifier(self.features(x))

```

Listing B.20: alpr/ocr/model.py

```

1  """
2  alpr/ocr/train.py
3  =====
4  Fase 3 - entrenament de CharCNN sobre el dataset sintètic.
5
6  Llegeix data/synthetic/{train,val}/ (format ImageFolder, 28x28
7  px, binàries).
8  Desa el millor checkpoint a models/char_cnn_best.pth.
9  """
10
11 import json
12 import logging
13 import random
14 from pathlib import Path
15
16 import matplotlib.pyplot as plt
17 import numpy as np
18 import torch
19 import torch.nn as nn
20 import torch.optim as optim
21 from torch.utils.data import DataLoader
22 from torchvision import datasets, transforms
23
24 from alpr import config
25 from .model import CharCNN
26
27 log = logging.getLogger(__name__)
28
29 # =====
30 # Càrrega de dades
31 # =====
32
33 def build_dataloaders(
34     data_dir: Path,
35     batch_size: int,
36 ) -> tuple[DataLoader, DataLoader, dict[str, int]]:
37     """
38     Construeix DataLoaders de train i val des de

```

```

39         data_dir/train i data_dir/val.
40     Retorna (train_loader, val_loader, class_to_idx).
41     """
42     transform = transforms.Compose([
43         transforms.Grayscale(num_output_channels=1),
44         transforms.ToTensor(),
45         transforms.Normalize((0.5,), (0.5,)),
46     ])
47     train_dir = data_dir / "train"
48     val_dir   = data_dir / "val"
49
50     if not train_dir.exists() or not any(train_dir.iterdir()):
51         raise FileNotFoundError(
52             f"No s'ha trobat el dataset a '{train_dir}'.\n"
53             "Executa primer scripts/build_ocr_dataset.py."
54         )
55
56     train_ds = datasets.ImageFolder(str(train_dir),
57                                     transform=transform)
58     val_ds   = datasets.ImageFolder(str(val_dir),
59                                     transform=transform)
59
60     # Verifica coherència amb class_to_idx.json
61     json_path = data_dir / "class_to_idx.json"
62     if json_path.exists():
63         saved = json.loads(json_path.read_text())
64         if saved != train_ds.class_to_idx:
65             raise ValueError(
66                 "class_to_idx del ImageFolder no coincideix
67                 amb class_to_idx.json.\n"
68                 "Regenera el dataset."
69             )
70
71     train_loader = DataLoader(train_ds, batch_size=batch_size,
72                               shuffle=True, num_workers=0,
73                               pin_memory=False)
74     val_loader   = DataLoader(val_ds, batch_size=batch_size,
75                               shuffle=False, num_workers=0,
76                               pin_memory=False)
77
78     log.info(f"Train: {len(train_ds)} imatges,
79              {len(train_ds.classes)} classes")
80     log.info(f"Val  : {len(val_ds)} imatges")
81
82     return train_loader, val_loader, train_ds.class_to_idx
83
84 # =====
85 # Bucles d'entrenament i avaluació
86 # =====

```

```

84 def train_one_epoch(
85     model: nn.Module,
86     loader: DataLoader,
87     criterion: nn.Module,
88     optimizer: optim.Optimizer,
89     device: torch.device,
90 ) -> tuple[float, float]:
91     """Una passada completa de train. Retorna (loss_mitja,
92         accuracy)."""
93     model.train()
94     total_loss = 0.0
95     n_correct = n_total = 0
96
97     for imgs, labels in loader:
98         imgs, labels = imgs.to(device), labels.to(device)
99         optimizer.zero_grad()
100         logits = model(imgs)
101         loss = criterion(logits, labels)
102         loss.backward()
103         optimizer.step()
104         total_loss += loss.item() * imgs.size(0)
105         n_correct += (logits.argmax(dim=1) ==
106             labels).sum().item()
107         n_total += imgs.size(0)
108
109     return total_loss / n_total, n_correct / n_total
110
111 def evaluate(
112     model: nn.Module,
113     loader: DataLoader,
114     criterion: nn.Module,
115     device: torch.device,
116 ) -> tuple[float, float]:
117     """Avaluació sense gradient. Retorna (loss_mitja,
118         accuracy)."""
119     model.eval()
120     total_loss = 0.0
121     n_correct = n_total = 0
122
123     with torch.no_grad():
124         for imgs, labels in loader:
125             imgs, labels = imgs.to(device), labels.to(device)
126             logits = model(imgs)
127             loss = criterion(logits, labels)
128             total_loss += loss.item() * imgs.size(0)
129             n_correct += (logits.argmax(dim=1) ==
130                 labels).sum().item()
131             n_total += imgs.size(0)
132
133     return total_loss / n_total, n_correct / n_total
134
135 # =====

```

```

134 # Corbes
135 # =====
136
137 def plot_curves(
138     history: dict[str, list[float]],
139     out_path: Path,
140     best_epoch: int,
141 ) -> None:
142     """Guarda figura amb les corbes de loss i accuracy (train
143         vs val)."""
144     epochs = range(1, len(history["train_loss"]) + 1)
145     fig, (ax_loss, ax_acc) = plt.subplots(1, 2, figsize=(11,
146         4))
147
148     ax_loss.plot(epochs, history["train_loss"], label="train")
149     ax_loss.plot(epochs, history["val_loss"], label="val")
150     ax_loss.axvline(best_epoch, color="gray", linestyle="--",
151         linewidth=0.8,
152         label=f"millor (ep. {best_epoch})")
153     ax_loss.set_title("Loss"); ax_loss.set_xlabel("Epoch");
154     ax_loss.legend()
155
156     ax_acc.plot(epochs, history["train_acc"], label="train")
157     ax_acc.plot(epochs, history["val_acc"], label="val")
158     ax_acc.axvline(best_epoch, color="gray", linestyle="--",
159         linewidth=0.8)
160     ax_acc.set_title("Accuracy"); ax_acc.set_xlabel("Epoch")
161     ax_acc.set_ylim(0, 1); ax_acc.legend()
162
163     plt.suptitle("Corbes d'entrenament CharCNN", fontsize=12)
164     plt.tight_layout()
165     plt.savefig(str(out_path), dpi=120)
166     plt.close(fig)
167     log.info(f"Corbes guardades a: {out_path}")
168
169 # =====
170
171 # Funció principal d'entrenament
172 # =====
173
174 def train(
175     data_dir: Path | None = None,
176     out_dir: Path | None = None,
177     epochs: int = 30,
178     batch_size: int = 64,
179     lr: float = 1e-3,
180     patience: int = 5,
181     seed: int = 42,
182 ) -> Path:
183     """

```

```

179 Entrena CharCNN i desa el millor checkpoint.
180 Retorna la ruta del checkpoint guardat.
181 """
182 if data_dir is None:
183     data_dir = config.DATA_SYNTHETIC_DIR
184 if out_dir is None:
185     out_dir = config.MODELS_DIR
186
187 random.seed(seed); np.random.seed(seed);
188     torch.manual_seed(seed)
189
190 if torch.cuda.is_available():
191     device = torch.device("cuda")
192 elif torch.backends.mps.is_available():
193     device = torch.device("mps")
194 else:
195     device = torch.device("cpu")
196 log.info(f"Device: {device}")
197
198 out_dir.mkdir(parents=True, exist_ok=True)
199
200 train_loader, val_loader, class_to_idx =
201     build_dataloaders(data_dir, batch_size)
202 n_classes = len(class_to_idx)
203
204 model = CharCNN(n_classes=n_classes).to(device)
205 criterion = nn.CrossEntropyLoss()
206 optimizer = optim.Adam(model.parameters(), lr=lr)
207
208 log.info(f"Model: CharCNN ({n_classes} classes)")
209 log.info(f"Paràmetres: {sum(p.numel() for p in
210     model.parameters()):,}")
211
212 history = {"train_loss": [], "train_acc": [],
213     "val_loss": [], "val_acc": []}
214 best_val_acc = -1.0
215 best_epoch = 1
216 patience_counter = 0
217 ckpt_path = out_dir / "char_cnn_best.pth"
218
219 log.info("-" * 60)
220 log.info(f"{'Epoch':>5} {'TrLoss':>7} {'TrAcc':>6}
221     {'VaLoss':>7} {'VaAcc':>6} {'*':>2}")
222 log.info("-" * 60)
223
224 for epoch in range(1, epochs + 1):
225     tr_loss, tr_acc = train_one_epoch(model, train_loader,
226         criterion, optimizer, device)
227     va_loss, va_acc = evaluate(model, val_loader,
228         criterion, device)
229
230     history["train_loss"].append(tr_loss)
231     history["train_acc"].append(tr_acc)
232     history["val_loss"].append(va_loss)

```

```

226     history["val_acc"].append(va_acc)
227
228     improved = va_acc > best_val_acc
229     log.info(f"{epoch:>5}   {tr_loss:>7.4f}   {tr_acc:>6.2%}"
230             "
231             f"{va_loss:>7.4f}   {va_acc:>6.2%}   {'ok' if
232                 improved else ''}")
233
234     if improved:
235         best_val_acc = va_acc; best_epoch = epoch;
236         patience_counter = 0
237         torch.save({
238             "epoch": epoch,
239             "model_state": model.state_dict(),
240             "val_acc": best_val_acc,
241             "n_classes": n_classes,
242             "class_to_idx": class_to_idx,
243         }, ckpt_path)
244     else:
245         patience_counter += 1
246         if patience_counter >= patience:
247             log.info(f"Early stopping a l'epoch {epoch}")
248             break
249
250     log.info(f"Millor val accuracy: {best_val_acc:.2%}   (epoch
251             {best_epoch})")
252     log.info(f"Checkpoint: {ckpt_path}")
253
254     plot_curves(history, out_dir / "training_curves.png",
255                 best_epoch)
256     return ckpt_path

```

Listing B.21: alpr/ocr/train.py

```

1  """
2  alpr/ocr/infer.py
3  =====
4  Fase 3 - inferència: càrrega del checkpoint i predicció de
5  caràcters.
6
7  API pública:
8      load_model(model_path)          -> (model, class_to_idx,
9      idx_to_class, device)
10     predict(char_img, model, ...)    -> (lletra, confiança)
11     accepta np.ndarray 28x28
12     predict_char(model, img_path, ...) -> (idx, confiança)
13     per a fitxers (compat. 04)
14 """
15
16 import logging
17 from pathlib import Path
18
19 import numpy as np
20 import torch

```

```

17 import torch.nn as nn
18 from PIL import Image
19 from torchvision import transforms
20
21 from alpr import config
22 from .model import CharCNN
23
24 log = logging.getLogger(__name__)
25
26 # Transform idèntic al de l'entrenament
27 _TRANSFORM = transforms.Compose([
28     transforms.Grayscale(num_output_channels=1),
29     transforms.Resize((config.OCR_INPUT_SIZE,
30         config.OCR_INPUT_SIZE)),
31     transforms.ToTensor(),
32     transforms.Normalize((0.5,), (0.5,)),
33 ])
34
35 # =====
36 # Càrrega del model
37 # =====
38
39 def load_model(
40     model_path: Path | None = None,
41 ) -> tuple[nn.Module, dict[str, int], dict[int, str],
42     torch.device]:
43     """
44     Carrega el checkpoint i reconstrueix CharCNN.
45     Retorna (model, class_to_idx, idx_to_class, device).
46     """
47     if model_path is None:
48         model_path = config.MODEL_CNN_PATH
49
50     if not model_path.exists():
51         raise FileNotFoundError(
52             f"No s'ha trobat el checkpoint a '{model_path}'.\n"
53             "Executa primer scripts/train_ocr.py."
54         )
55
56     if torch.cuda.is_available():
57         device = torch.device("cuda")
58     elif torch.backends.mps.is_available():
59         device = torch.device("mps")
60     else:
61         device = torch.device("cpu")
62
63     ckpt = torch.load(str(model_path),
64         map_location=device, weights_only=False)
65     n_classes = ckpt["n_classes"]
66     class_to_idx = ckpt["class_to_idx"]
67     idx_to_class = {v: k for k, v in class_to_idx.items()}

```



```

66
67     model = CharCNN(n_classes=n_classes)
68     model.load_state_dict(ckpt["model_state"])
69     model.to(device)
70     model.eval()
71
72     val_acc = ckpt.get("val_acc")
73     if val_acc is not None:
74         log.info(f"Model carregat: {n_classes} classes |
75                 val_acc = {val_acc:.2%}")
76     else:
77         log.info(f"Model carregat: {n_classes} classes")
78
79     return model, class_to_idx, idx_to_class, device
80
81 # =====
82 # Inferència
83 # =====
84
85 def _to_tensor(img: np.ndarray | Image.Image) -> torch.Tensor:
86     """Converteix np.ndarray uint8 o PIL Image al tensor
87     esperat pel model."""
88     if isinstance(img, np.ndarray):
89         img = Image.fromarray(img)
90     return _TRANSFORM(img).unsqueeze(0)
91
92 def predict(
93     char_img: np.ndarray,
94     model: nn.Module,
95     idx_to_class: dict[int, str],
96     device: torch.device,
97     threshold: float | None = None,
98 ) -> tuple[str, float]:
99     """
100     Prediu el caràcter d'una imatge np.ndarray 28x28 binària.
101
102     Retorna (lletre, confiança_softmax).
103     Si confiança < threshold retorna ('?', confiança).
104     """
105     if threshold is None:
106         threshold = config.CONF_THRESHOLD
107
108     tensor = _to_tensor(char_img).to(device)
109     with torch.no_grad():
110         probs = torch.softmax(model(tensor), dim=1).squeeze()
111
112     idx = int(probs.argmax().item())
113     conf = float(probs[idx].item())
114     char = idx_to_class.get(idx, "?")
115

```

```

116     if conf < threshold:
117         return "?", conf
118     return char, conf
119
120
121 def predict_char(
122     model: nn.Module,
123     img_path: Path,
124     device: torch.device,
125 ) -> tuple[int, float]:
126     """
127     Prediu el caràcter d'un fitxer d'imatge.
128     Retorna (idx_predit, confiança_softmax).
129     Compatible amb la firma original de 04_evaluate_real.py.
130     """
131     img = Image.open(img_path).convert("L")
132     tensor = _TRANSFORM(img).unsqueeze(0).to(device)
133     with torch.no_grad():
134         probs = torch.softmax(model(tensor), dim=1).squeeze()
135         idx = int(probs.argmax().item())
136         conf = float(probs[idx].item())
137     return idx, conf
138
139
140 def filter_by_confidence(
141     preds: list[dict],
142     threshold: float | None = None,
143 ) -> tuple[list[dict], list[dict]]:
144     """
145     Divideix prediccions en retingudes (conf >= threshold) i
146     descartades.
147     Cada element de preds ha de tenir la clau 'confidence'.
148     """
149     if threshold is None:
150         threshold = config.CONF_THRESHOLD
151     retained = [p for p in preds if p["confidence"] >=
152                 threshold]
153     discarded = [p for p in preds if p["confidence"] <
154                 threshold]
155     return retained, discarded

```

Listing B.22: alpr/ocr/infer.py

```

1  """
2  alpr/ocr/eval_real.py
3  =====
4  Fase 3 - avaluació del model CharCNN sobre caràcters REALS
5      segmentats.
6
7  Opera sobre els PNGs que el segmentador deixa a 'data/chars/'
8  ({stem}_box{n}_char{i}.png), els classifica amb la CNN,
9      filtra els de baixa
10     confiança i compara la seqüència resultant amb el ground truth
11     de 'data/raw/'

```

```

9 via distància d'edició (Levenshtein) - robusta a
    desalineacions de longitud.
10
11 Genera: resum de mètriques (char/plate accuracy ED,
    posicional, gap sintètic→real),
12 'models/eval_real_results.csv' i
    'models/confusion_matrix_real.png'.
13
14 Reutilitza 'load_model', 'predict_char' i
    'filter_by_confidence' d' 'alpr.ocr.infer',
15 i el parseig d' 'alpr.common.annotations' (cap lògica
    duplicada).
16
17 Ús
18 --
19     python -m alpr.ocr.eval_real
20     python -m alpr.ocr.eval_real --conf-threshold 0.6
21     python -m alpr.ocr.eval_real --strict-length
22 """
23
24 from __future__ import annotations
25
26 import argparse
27 import csv
28 import logging
29 from collections import Counter, defaultdict
30 from pathlib import Path
31
32 import numpy as np
33 import torch
34
35 from alpr import config
36 from alpr.common.annotations import load_annotations,
    parse_char_filename
37 from alpr.ocr.infer import load_model, predict_char,
    filter_by_confidence
38
39 logging.basicConfig(level=logging.INFO, format="%(levelname)s
    %(message)s")
40 log = logging.getLogger(__name__)
41
42
43 # =====
44 # Ground truth i distància d'edició
45 # =====
46
47 def parse_gt(gt_dir: Path, stem: str) -> list[str]:
48     """Retorna les matrícules GT de '{gt_dir}/{stem}.txt'
        (majúscules)."""
49     return [a["plate"] for a in load_annotations(gt_dir /
        f"{stem}.txt")]
50

```

```

51
52 def edit_distance(s1: str, s2: str) -> int:
53     """
54     Distància d'edició (inserció/eliminació/substitució) entre
55     dues cadenes.
56     Programació dinàmica amb O(min(m, n)) d'espai.
57     """
58     if len(s1) < len(s2):
59         s1, s2 = s2, s1
60     m, n = len(s1), len(s2)
61     if n == 0:
62         return m
63     prev = list(range(n + 1))
64     for i, c1 in enumerate(s1, 1):
65         curr = [i] + [0] * n
66         for j, c2 in enumerate(s2, 1):
67             cost = 0 if c1 == c2 else 1
68             curr[j] = min(curr[j - 1] + 1, prev[j] + 1, prev[j - 1] + cost)
69         prev = curr
70     return prev[n]
71
72 # =====
73 # Agrupació de caràcters per ROI
74 # =====
75
76 def group_files(chars_dir: Path, gt_dir: Path) ->
77     tuple[list[dict], int]:
78     """
79     Agrupa els PNGs per '{stem}_box{n}' i associa el GT del
80     stem. No comprova
81     longitud (tots els grups amb GT es retornen). Retorna
82     (groups, n_sense_gt).
83     """
84     raw: dict[str, list] = defaultdict(list)
85     for fpath in sorted(chars_dir.glob("*.png")):
86         stem, box_n, char_i = parse_char_filename(fpath)
87         if stem is None:
88             log.warning(f"Nom no reconegut, ignorat:
89                         {fpath.name}")
90             continue
91         raw[f"{stem}_box{box_n}"].append((char_i, fpath, stem,
92                                           box_n))
93
94     groups: list[dict] = []
95     n_sense_gt = 0
96     for group_key, entries in sorted(raw.items()):
97         entries.sort(key=lambda e: e[0])
98         stem, box_n = entries[0][2], entries[0][3]
99         plates_gt = parse_gt(gt_dir, stem)
100         if not plates_gt:

```

```

96         n_sense_gt += 1
97         continue
98     groups.append({
99         "group_key": group_key,
100         "stem": stem,
101         "box": box_n,
102         "entries": [(e[0], e[1]) for e in entries],
103         "plates_gt": plates_gt,
104     })
105     return groups, n_sense_gt
106
107
108 # =====
109 # Avaluació d'un grup
110 # =====
111
112 def evaluate_group(
113     model,
114     group: dict,
115     threshold: float,
116     idx_to_class: dict[int, str],
117     device,
118     strict_length: bool,
119 ) -> dict | None:
120     """Inferència → filtratge per confiança → comparació amb
121     el millor GT (ED)."""
122     all_preds: list[dict] = []
123     for char_idx, img_path in group["entries"]:
124         pred_idx, conf = predict_char(model, img_path, device)
125         all_preds.append({
126             "char_idx": char_idx,
127             "pred": idx_to_class.get(pred_idx, "?"),
128             "confidence": conf,
129             "filtered_out": False,
130         })
131
132     retained, discarded = filter_by_confidence(all_preds,
133         threshold)
134     for p in discarded:
135         p["filtered_out"] = True
136
137     pred_str = "".join(p["pred"] for p in retained)
138     best_gt = min(group["plates_gt"], key=lambda g:
139         edit_distance(pred_str, g))
140     gt_len = len(best_gt)
141     ed = edit_distance(pred_str, best_gt)
142
143     if strict_length and len(retained) != gt_len:
144         return None
145
146     positional_pairs: list[dict] = []
147     if len(retained) == gt_len:

```

```

145         for pred_info, gt_char in zip(retained, best_gt):
146             positional_pairs.append({
147                 "gt": gt_char,
148                 "pred": pred_info["pred"],
149                 "correct": gt_char == pred_info["pred"],
150             })
151
152     return {
153         "group_key": group["group_key"],
154         "stem": group["stem"],
155         "box": group["box"],
156         "gt": best_gt,
157         "pred_str": pred_str,
158         "n_segmented": len(all_preds),
159         "n_after_filter": len(retained),
160         "n_discarded": len(discarded),
161         "gt_length": gt_len,
162         "edit_distance": ed,
163         "plate_match": ed == 0,
164         "lengths_match": len(retained) == gt_len,
165         "positional_pairs": positional_pairs,
166         "all_preds": all_preds,
167     }
168
169
170 # =====
171 # Mètriques agregades
172 # =====
173
174 def compute_metrics(results: list[dict]) -> dict:
175     """Agrega char/plate accuracy (ED) i accuracy posicional
176     sobre els grups."""
177     if not results:
178         return {}
179
180     n_groups = len(results)
181     total_ed = sum(r["edit_distance"] for r in results)
182     total_gt_len = sum(r["gt_length"] for r in results)
183     n_match = sum(r["plate_match"] for r in results)
184
185     pos_pairs = [p for r in results for p in
186                  r["positional_pairs"]]
187     char_acc_pos = (
188         sum(p["correct"] for p in pos_pairs) / len(pos_pairs)
189         if pos_pairs else None
190     )
191
192     return {
193         "n_groups": n_groups,
194         "total_ed": total_ed,
195         "total_gt_len": total_gt_len,
196         "n_plate_match": n_match,

```

```

194         "char_acc_ed": max(0.0, 1.0 - total_ed /
195                             total_gt_len) if total_gt_len else 0.0,
196         "plate_acc_ed": n_match / n_groups,
197         "char_acc_pos": char_acc_pos,
198         "n_pos_groups": sum(1 for r in results if
199                             r["lengths_match"]),
200         "n_pos_pairs": len(pos_pairs),
201     }
202
203 def plot_confusion(y_true, y_pred, classes, out_path: Path) ->
204     list[tuple[str, str, int]]:
205     """Guarda la matriu de confusió (normalitzada per fila) i
206     retorna el top-10 fora diagonal."""
207     import matplotlib
208     matplotlib.use("Agg")
209     import matplotlib.pyplot as plt
210     from sklearn.metrics import confusion_matrix
211
212     cm = confusion_matrix(y_true, y_pred, labels=classes)
213     n = len(classes)
214     with np.errstate(divide="ignore", invalid="ignore"):
215         cm_norm = np.where(cm.sum(axis=1, keepdims=True) > 0,
216                             cm / cm.sum(axis=1, keepdims=True),
217                             0.0)
218
219     fig, ax = plt.subplots(figsize=(14, 12))
220     im = ax.imshow(cm_norm, interpolation="nearest",
221                    cmap="Blues", vmin=0, vmax=1)
222     plt.colorbar(im, ax=ax, label="Fracció per fila (classe
223                    real)")
224     ax.set_xticks(range(n)); ax.set_yticks(range(n))
225     ax.set_xticklabels(classes, fontsize=6, rotation=90)
226     ax.set_yticklabels(classes, fontsize=6)
227     ax.set_xlabel("Caràcter predit"); ax.set_ylabel("Caràcter
228                    real")
229     ax.set_title("Matriu de confusió - caràcters reals (grups
230                    alineats, normalitzada per fila)")
231     plt.tight_layout()
232     plt.savefig(str(out_path), dpi=120)
233     plt.close(fig)
234     log.info(f"Matriu de confusió guardada a: {out_path}")
235
236     off = [(classes[i], classes[j], int(cm[i, j]))
237            for i in range(n) for j in range(n) if i != j and
238                cm[i, j] > 0]
239     off.sort(key=lambda x: -x[2])
240     return off[:10]
241
242 # =====
243 # CSV
244 # =====

```

```

237
238 def write_csv(results: list[dict], csv_path: Path) -> None:
239     """Desa un CSV amb una fila per caràcter (inclou els
        filtrats per confiança)."""
240     fieldnames = ["stem", "box", "char_idx", "gt", "pred",
        "confidence", "correct",
241                 "filtered_out", "n_segmented",
        "n_after_filter", "gt_length",
242                 "edit_distance", "n_discarded_lowconf"]
243     with open(csv_path, "w", newline="", encoding="utf-8") as
        f:
244         writer = csv.DictWriter(f, fieldnames=fieldnames)
245         writer.writeheader()
246         for r in results:
247             retained = [p for p in r["all_preds"] if not
                p["filtered_out"]]
248             gt_map: dict[int, str] = {}
249             if r["lengths_match"]:
250                 for pred_info, gt_char in zip(retained,
                    r["gt"]):
251                     gt_map[id(pred_info)] = gt_char
252             for p in r["all_preds"]:
253                 if r["lengths_match"] and not
                    p["filtered_out"]:
254                     gt_char = gt_map.get(id(p), "?")
255                     correct = gt_char == p["pred"]
256                 else:
257                     gt_char, correct = "?", False
258                 writer.writerow({
259                     "stem": r["stem"], "box": r["box"],
                        "char_idx": p["char_idx"],
260                     "gt": gt_char, "pred": p["pred"],
261                     "confidence": f"{p['confidence']:.4f}",
                        "correct": correct,
262                     "filtered_out": p["filtered_out"],
263                     "n_segmented": r["n_segmented"],
                        "n_after_filter": r["n_after_filter"],
264                     "gt_length": r["gt_length"],
                        "edit_distance": r["edit_distance"],
265                     "n_discarded_lowconf": r["n_discarded"],
266                 })
267             log.info(f"CSV guardat a: {csv_path}")
268
269
270 # =====
        # Punt d'entrada
271 # =====
272
273
274 def main() -> None:
275     parser = argparse.ArgumentParser(description="Avalua
        CharCNN sobre caràcters reals.")

```



```

276 parser.add_argument("--chars-dir", type=Path,
277                     default=config.DATA_CHARS_DIR,
278                     help=f"Directori amb els PNGs
279                          segmentats (default:
280                          {config.DATA_CHARS_DIR})")
281 parser.add_argument("--gt-dir", type=Path,
282                     default=config.DATA_RAW_DIR,
283                     help=f"Directori amb els .txt de GT
284                          (default: {config.DATA_RAW_DIR})")
285 parser.add_argument("--model", type=Path, default=None,
286                     help=f"Checkpoint (default:
287                     {config.MODEL_CNN_PATH})")
288 parser.add_argument("--out-dir", type=Path,
289                     default=config.MODELS_DIR,
290                     help=f"On desar CSV i figura (default:
291                     {config.MODELS_DIR})")
292 parser.add_argument("--conf-threshold", type=float,
293                     default=config.CONF_THRESHOLD,
294                     help=f"Llindar de confiança (default:
295                     {config.CONF_THRESHOLD})")
296 parser.add_argument("--strict-length", action="store_true",
297                     help="Descarta grups on la longitud
298                          filtrada ≠ longitud GT")
299
300 args = parser.parse_args()
301
302 args.out_dir.mkdir(parents=True, exist_ok=True)
303
304 groups, n_sense_gt = group_files(args.chars_dir,
305                                  args.gt_dir)
306 if not groups:
307     log.error(f"Cap grup amb GT a '{args.chars_dir}'.
308              Executa primer el segmentador.")
309     return
310 log.info(f"Grups amb GT: {len(groups)} (sense GT:
311         {n_sense_gt})")
312
313 model, class_to_idx, idx_to_class, device =
314     load_model(args.model)
315
316 results: list[dict] = []
317 n_strict = 0
318 for g in groups:
319     r = evaluate_group(model, g, args.conf_threshold,
320                       idx_to_class, device,
321                       args.strict_length)
322
323     if r is None:
324         n_strict += 1
325     else:
326         results.append(r)
327 if not results:
328     log.error("Tots els grups descartats
329              (--strict-length).")
330 return

```

```

313 metrics = compute_metrics(results)
314
315 classes_sorted = sorted(class_to_idx, key=lambda c:
316                          class_to_idx[c])
317 pos_pairs = [p for r in results for p in
318              r["positional_pairs"]]
319 top10: list = []
320 if pos_pairs:
321     top10 = plot_confusion([p["gt"] for p in pos_pairs],
322                          [p["pred"] for p in pos_pairs],
323                          classes_sorted, args.out_dir /
324                          "confusion_matrix_real.png")
325
326 write_csv(results, args.out_dir / "eval_real_results.csv")
327
328 # --- Resum -----
329 -----
330 print("\n" + "=" * 60)
331 print("AVALUACIÓ SOBRE CARÀCTERS REALS")
332 print("=" * 60)
333 print(f" Grups avaluats : {metrics['n_groups']}"
334       + (f" (descartats strict: {n_strict})" if n_strict
335          else ""))
336 print(f" Char Accuracy (ED) :
337       {metrics['char_acc_ed']:.2%}"
338       + f" (ED={metrics['total_ed']}, chars
339          GT={metrics['total_gt_len']})")
340 print(f" Plate Accuracy (ED=0) :
341       {metrics['plate_acc_ed']:.2%}"
342       + f"
343          ({metrics['n_plate_match']}/{metrics['n_groups']}
344          perfectes)")
345 if metrics["char_acc_pos"] is not None:
346     print(f" Char Accuracy posicional:
347           {metrics['char_acc_pos']:.2%}"
348           + f" ({metrics['n_pos_pairs']} caràcters,
349              {metrics['n_pos_groups']} grups alineats)")
350 if top10:
351     print("\n Top parells de confusió (real → predict):")
352     for gt_c, pred_c, cnt in top10:
353         print(f" {gt_c} → {pred_c} ({cnt})")
354 print("=" * 60 + "\n")
355
356 if __name__ == "__main__":
357     main()

```

Listing B.23: alpr/ocr/eval_real.py

B.7 Fase 3b — Reader

```
1 """
```

```

2 alpr/reader/reader.py
3 =====
4 Fase 3b - Reader: munta el string final de la matrícula a
    partir dels caràcters
5 predits per l'OCR.
6
7 IMPORTANT - aquest dataset NO segueix cap patró. Les
    matrícules poden ser de
8 qualsevol país europeu (Sèrbia, Eslovènia, Turquia,
    Espanya...) i contenen lletres
9 (A-Z) i dígit (0-9) en MAJÚSCULES, de 5 a 9 caràcters, en
    ordre **totalment
10 aleatori**. Cap posició té un tipus esperat (lletra o dígit).
11
12 Conseqüència de disseny: NO es pot aplicar cap correcció
    contextual de confusions
13 (0↔0, S↔5, I↔1...), perquè no hi ha manera de saber en quina
    direcció corregir sense
14 un patró posicional - i corregir a cegues introduiria errors
    nous. El reader, per
15 tant, es limita a **netejar i concatenar** la millor predicció
    de l'OCR.
16 """
17
18 from __future__ import annotations
19
20 import re
21
22
23 def read_plate(chars: list[str]) -> str:
24     """
25     Munta la matrícula final: concatena els caràcters, passa a
        majúscules i
26     elimina '?' (baixa confiança) i qualsevol símbol no
        alfanumèric.
27
28     No aplica cap correcció contextual: el dataset no té
        format fix (veure la
29     capçalera del mòdul), així que la predicció de l'OCR es
        retorna tal qual,
30     només netejada.
31     """
32     text = "".join(chars).upper()
33     return re.sub(r"[^A-Z0-9]", "", text)

```

Listing B.24: alpr/reader/reader.py

B.8 Orquestrador end-to-end

```

1 """
2 main.py
3 =====

```

```

4 Pipeline end-to-end de detecció i lectura de matrícules (ALPR).
5
6 Flux complet per imatge:
7     imatge BGR
8     +- Fase 1 · detector      → retalls candidats (màxim recall)
9     +- Fase 2 · segmenter    → caràcters 28×28 per cada retall
10    vàlid
11    +- Fase 3 · ocr/infer     → lletra + confiança per cada
12    caràcter
13    +- Fase 4 · reader        → matrícula final (correcció 0→0,
14    I→1...)
15
16 Ús
17 --
18     # Una sola imatge
19     python main.py --input data/raw/foto.jpg
20
21     # Tot un directori
22     python main.py --input data/raw/
23
24     # Desar resultats en CSV
25     python main.py --input data/raw/ --output
26     output/results.csv
27
28     # Mode debug (mostra cada etapa per pantalla)
29     python main.py --input data/raw/foto.jpg --debug
30 """
31
32 import argparse
33 import csv
34 import logging
35 import sys
36 import time
37 from pathlib import Path
38
39 import cv2
40 import numpy as np
41
42 from alpr import config
43 from alpr.common.io import load_image, iter_images
44 from alpr.detector.detector import detect_boxes, detect_debug
45 from alpr.segmenter.segmenter import segment, segmenta_caixa
46 from alpr.ocr.infer import load_model, predict
47 from alpr.reader.reader import read_plate
48
49 # --- Logger -----
50
51 logging.basicConfig(
52     level=logging.INFO,
53     format="%(asctime)s %(levelname)-7s %(message)s",
54     datefmt="%H:%M:%S",
55 )
56 log = logging.getLogger(__name__)

```

```

53
54
55 # =====
56 # Pipeline per a una sola imatge
57 # =====
58
59 def process_image(
60     img_bgr: np.ndarray,
61     model,
62     idx_to_class: dict,
63     device,
64     debug: bool = False,
65 ) -> list[dict]:
66     """
67     Executa el pipeline complet sobre una imatge BGR.
68
69     Retorna llista de dicts amb:
70     { 'box': (x,y,w,h), 'plate': str, 'n_chars': int,
71       'chars': list[str], 'confs': list[float], 'mean_conf':
72         float }
73
74     Llista buida si no es detecta cap matrícula vàlida.
75     """
76     results = []
77
78     # -- Fase 1: Detecció -----
79     if debug:
80         det = detect_debug(img_bgr)
81         boxes = det["boxes"]
82         log.debug(f" Detector: {det['n_total']} components -> {len(boxes)} candidats")
83     else:
84         boxes = detect_boxes(img_bgr)
85
86     if not boxes:
87         return results
88
89     # -- Fases 2-4: per cada candidat -----
90     for box in boxes:
91         x, y, w, h = box
92         roi = img_bgr[y : y + h, x : x + w]
93
94         if debug:
95             seg = segmenta_caixa(roi)
96             chars_imgs = seg["chars"]
97             accepted = seg["accepted"]
98             if not accepted:
99                 log.debug(f" ROI ({x},{y},{w},{h}) rebutjada: {seg['rejection_reason']}")
100                 continue

```

```

100     else:
101         chars_imgs = segment(roi)
102         if not chars_imgs:
103             continue
104
105     # Fase 3 - OCR per caràcter
106     char_preds = [
107         predict(c, model, idx_to_class, device)
108         for c in chars_imgs
109     ]
110     chars = [p[0] for p in char_preds]
111     confs = [p[1] for p in char_preds]
112
113     # Filtra '?' (baixa confiança)
114     valid = [(ch, cf) for ch, cf in zip(chars, confs) if
115              ch != "?"]
116     if not valid:
117         continue
118     chars_v = [v[0] for v in valid]
119     confs_v = [v[1] for v in valid]
120
121     # Fase 4 - Reader (correcció contextual)
122     plate = read_plate(chars_v)
123     mean_conf = float(np.mean(confs_v))
124
125     results.append({
126         "box": box,
127         "plate": plate,
128         "n_chars": len(chars_v),
129         "chars": chars_v,
130         "confs": confs_v,
131         "mean_conf": mean_conf,
132     })
133
134     return results
135
136 # =====
137 # Visualització debug
138 # =====
139
140 def draw_results(
141     img_bgr: np.ndarray,
142     results: list[dict],
143 ) -> np.ndarray:
144     """Dibuixa bboxes i text de matrícula sobre la imatge.
145     Retorna còpia."""
146     out = img_bgr.copy()
147     for r in results:
148         x, y, w, h = r["box"]
149         plate = r["plate"]
150         conf = r["mean_conf"]

```

```

150
151     color = (0, 200, 50) # verd
152     cv2.rectangle(out, (x, y), (x + w, y + h), color, 2)
153
154     label = f"{plate} ({conf:.0%})"
155     (tw, th), _ = cv2.getTextSize(label,
156                                   cv2.FONT_HERSHEY_SIMPLEX, 0.6, 1)
157     cv2.rectangle(out, (x, y - th - 6), (x + tw + 4, y),
158                   color, -1)
159     cv2.putText(out, label, (x + 2, y - 4),
160                  cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 0),
161                  1)
162
163     return out
164
165 # =====
166 # Bucle principal
167 # =====
168
169 def run(
170     input_path: Path,
171     output_csv: Path | None,
172     model_path: Path | None,
173     debug: bool,
174     show: bool,
175 ) -> None:
176     """
177     Executa el pipeline sobre input_path (imatge o directori).
178     Opcionalment guarda els resultats en output_csv.
179     """
180     # -- Càrrega del model OCR -----
181     log.info("Carregant model OCR...")
182     try:
183         model, _class_to_idx, idx_to_class, device =
184             load_model(model_path)
185     except FileNotFoundError as e:
186         log.error(str(e))
187         log.error(
188             "Entrena primer el model amb:\n"
189             "    python scripts/train_ocr.py"
190         )
191         sys.exit(1)
192
193     # -- Llista d'imatges a processar -----
194
195     if input_path.is_file():
196         images = [(input_path, load_image(input_path))]
197     elif input_path.is_dir():
198         images = list(iter_images(input_path))
199     else:

```

```

196         log.error(f"Ruta no vàlida: '{input_path}'")
197         sys.exit(1)
198
199     if not images:
200         log.warning(f"No s'han trobat imatges a
201             '{input_path}'")
202         return
203
204     log.info(f"Processant {len(images)} imatge(s) ...")
205     log.info("-" * 60)
206
207     # -- Preparació CSV -----
208     csv_rows: list[dict] = []
209
210     # -- Bucle d'imatges -----
211     t0_global = time.time()
212
213     for img_path, img_bgr in images:
214         t0 = time.time()
215         results = process_image(img_bgr, model, idx_to_class,
216             device, debug=debug)
217         elapsed = time.time() - t0
218
219         if results:
220             for r in results:
221                 plate = r["plate"]
222                 conf = r["mean_conf"]
223                 log.info(f"    {img_path.name:<30}    {plate:<10}
224                     conf={conf:.0%}    ({elapsed*1000:.0f} ms)")
225                 csv_rows.append({
226                     "file":      img_path.name,
227                     "plate":     plate,
228                     "n_chars":   r["n_chars"],
229                     "mean_conf": f"{conf:.4f}",
230                     "box":       str(r["box"]),
231                 })
232         else:
233             log.info(f"    {img_path.name:<30}    [sense detecció]
234                 ({elapsed*1000:.0f} ms)")
235             csv_rows.append({
236                 "file":      img_path.name,
237                 "plate":     "",
238                 "n_chars":   0,
239                 "mean_conf": "",
240                 "box":       "",
241             })
242
243     # Visualització opcional. En entorns sense display
244     # (headless/CT) cv2.imshow
245     # llança cv2.error: el capturem perquè no aturi el
246     # processament del lot.
247     if show or debug:

```



```

242         annotated = draw_results(img_bgr, results)
243     try:
244         cv2.imshow(img_path.name, annotated)
245         key = cv2.waitKey(0 if debug else 500)
246         cv2.destroyAllWindows()
247         if key == ord("q"):
248             log.info("Interromput per l'usuari.")
249             break
250     except cv2.error:
251         log.warning("Sense display disponible: s'omet
                la visualització (--show/--debug).")
252
253     total = time.time() - t0_global
254     n_detected = sum(1 for r in csv_rows if r["plate"])
255     log.info("-" * 60)
256     log.info(f"Resultat: {n_detected}/{len(images)} imatges
                amb matrícula detectada")
257     log.info(f"Temps total: {total:.1f} s
                ({total/len(images)*1000:.0f} ms/imatge)")
258
259     # -- Desa CSV -----
260     if output_csv and csv_rows:
261         output_csv.parent.mkdir(parents=True, exist_ok=True)
262         with output_csv.open("w", newline="",
263                               encoding="utf-8") as f:
264             writer = csv.DictWriter(
265                 f, fieldnames=["file", "plate", "n_chars",
266                                "mean_conf", "box"]
267             )
268             writer.writeheader()
269             writer.writerows(csv_rows)
270             log.info(f"Resultats desats a: {output_csv}")
271
272     # =====
273     # Punt d'entrada CLI
274     # =====
275
276 def main() -> None:
277     parser = argparse.ArgumentParser(
278         description="Pipeline ALPR: detecció + segmentació +
279                     OCR de matrícules.",
280         formatter_class=argparse.RawTextHelpFormatter,
281     )
282     parser.add_argument(
283         "--input", required=True,
284         help="Imatge .jpg/.png o directori amb imatges.",
285     )
286     parser.add_argument(
287         "--output", default=None,

```

```

286         help="Fitxer CSV de sortida (opcional). Ex:
287             output/results.csv",
288     )
289     parser.add_argument(
290         "--model", default=None,
291         help=f"Ruta al checkpoint OCR (default:
292             {config.MODEL_CNN_PATH})",
293     )
294     parser.add_argument(
295         "--debug", action="store_true",
296         help="Mode debug: mostra etapes intermèdies i obre
297             finestres OpenCV.",
298     )
299     parser.add_argument(
300         "--show", action="store_true",
301         help="Mostra el resultat visual de cada imatge.",
302     )
303     parser.add_argument(
304         "--verbose", action="store_true",
305         help="Activa logs de nivell DEBUG.",
306     )
307     args = parser.parse_args()
308
309     if args.verbose or args.debug:
310         logging.getLogger().setLevel(logging.DEBUG)
311
312     run(
313         input_path = Path(args.input),
314         output_csv = Path(args.output) if args.output else
315             None,
316         model_path = Path(args.model) if args.model else None,
317         debug       = args.debug,
318         show        = args.show,
319     )
320
321 if __name__ == "__main__":
322     main()

```

Listing B.25: main.py